



# Smart Door System

Im Studiengang

**Elektrotechnik**

Am Campus Gelsenkirchen

Betreut von:

**Prof. Dr. Christos Georgiadis**

Autor:

**Demanou Ydriss Armel      202523503**

Stand: 18.02.2026

## Inhaltsverzeichnis

|                                                         |    |
|---------------------------------------------------------|----|
| <b>1. Einleitung</b>                                    | 6  |
| 1.1 Motivation                                          | 7  |
| 1.2 Problemstellung                                     | 7  |
| 1.3 Zielsetzung der Arbeit                              | 8  |
| <b>2. Grundlagen und technische Konzepte</b>            | 9  |
| 2.1 Smart-Home-Systeme                                  | 9  |
| 2.2 Raspberry Pi als Embedded System                    | 10 |
| 2.3 GPIO-Schnittstellen                                 | 11 |
| 2.4 Kommunikationsprotokolle (I <sup>2</sup> C und SPI) | 12 |
| 2.5 Sensor- und Aktorprinzipien                         | 13 |
| <b>3. Entwicklungsverlauf</b>                           | 14 |
| 3.1 Erste Phase: ESP32-CAM                              | 14 |
| 3.2 Umstieg auf Raspberry Pi                            | 15 |
| <b>4. Hardwarekonzept und Komponenten</b>               | 17 |
| 4.1 Systemübersicht                                     | 17 |
| 4.2 Raspberry Pi                                        | 18 |
| 4.3 PIR-Bewegungssensor                                 | 18 |
| 4.4 LDR mit ADC (ADS7830)                               | 19 |
| 4.4.1 Funktionsprinzip des LDR                          | 19 |
| 4.5 RFID-Modul (MFRC522)                                | 22 |

## Smart Door System

|                                                         |           |
|---------------------------------------------------------|-----------|
| 4.5.1 Grundprinzip der RFID-Technologie .....           | 22        |
| 4.5.2 Funktionsweise des MFRC522 .....                  | 23        |
| 4.6 Servomotor .....                                    | 24        |
| 4.7 RGB-LED .....                                       | 26        |
| 4.8 Buzzer .....                                        | 27        |
| 4.9 LCD1602-Display .....                               | 28        |
| 4.10 Kamera-Modul .....                                 | 30        |
| <b>5. Schaltungsdesign und Verdrahtung .....</b>        | <b>32</b> |
| 5.1 Stromversorgungskonzept .....                       | 33        |
| 5.2 Sicherheitshinweise bei der Verdrahtung .....       | 33        |
| <b>6. Softwarearchitektur und Implementierung .....</b> | <b>35</b> |
| 6.1 Systemarchitektur .....                             | 35        |
| 6.2 Ereignisbasierte Logik .....                        | 35        |
| 6.3 Multithreading .....                                | 36        |
| 6.4 Webserver-Implementierung .....                     | 36        |
| 6.5 Telegram-Integration .....                          | 38        |
| <b>7. Funktionsablauf des Systems .....</b>             | <b>39</b> |
| 7.1 Bewegungserkennung .....                            | 39        |
| 7.2 Tag-/Nacht-Entscheidung .....                       | 39        |
| 7.3 RFID-Zugangskontrolle .....                         | 39        |
| 7.4 Klingel-Logik .....                                 | 40        |

|                                                      |           |
|------------------------------------------------------|-----------|
| 7.5 Türsteuerung .....                               | 40        |
| <b>8. Mini-Haus als Demonstrationsmodell .....</b>   | <b>41</b> |
| 8.1 Mechanischer Aufbau .....                        | 41        |
| 8.2 Integration der Elektronik .....                 | 42        |
| 8.3 Herausforderungen beim Aufbau .....              | 43        |
| <b>9. Testphase und Ergebnisse .....</b>             | <b>44</b> |
| 9.1 Funktionstests der Sensoren .....                | 44        |
| 9.2 Stabilität des Systems .....                     | 45        |
| 9.3 Bewertung der Zuverlässigkeit .....              | 46        |
| <b>10. Sicherheits- und Datenschutzaspekte .....</b> | <b>47</b> |
| <b>11. Fazit .....</b>                               | <b>48</b> |
| <b>12. Ausblick und mögliche Erweiterungen .....</b> | <b>49</b> |
| Literaturverzeichnis .....                           | 50        |
| <b>13. Anhang .....</b>                              | <b>52</b> |
| Anhang1: Konfiguration & GPIO-Zuordnung .....        | 53        |
| Anhang2: ADS7830 lesen (LDR → Tag/Nacht) .....       | 54        |
| Anhang3: Telegram Nachricht + Foto senden .....      | 55        |
| Anhang4: Hauptlogik .....                            | 56        |
| Anhang5: Flask Webserver .....                       | 57        |

## Smart Door System

|                                                                                                                                    |    |
|------------------------------------------------------------------------------------------------------------------------------------|----|
| Abbildung 1: ESP32-CAM (ESP32, 2023) .....                                                                                         | 15 |
| Abbildung 2: Raspberry Pi 4 Model B (Raspberry pi Foundation, 2023).....                                                           | 16 |
| Abbildung 3: Systemarchitektur des Smart-Door-Systems mit allen Komponenten .....                                                  | 17 |
| Abbildung 4: Vergleich der Technologien.....                                                                                       | 19 |
| Abbildung 5: PIR-Sensor (Hill, 2015).....                                                                                          | 19 |
| Abbildung 6: ADS7830 Analog-Digital-Wandler mit I <sup>2</sup> C-Schnittstelle (Texas Instruments (ADS7830 Datasheet), 2023) ..... | 20 |
| Abbildung 7: LDR (lichtabhängiger Widerstand) (Hill, 2015) .....                                                                   | 21 |
| Abbildung 8: TSL2561 digitaler Lichtsensor .....                                                                                   | 22 |
| Abbildung 9: BH1750 digitaler Lux-Sensor mit I <sup>2</sup> C-Schnittstelle .....                                                  | 22 |
| Abbildung 10: MFRC522 RFID-Modul mit SPI-Schnittstelle.....                                                                        | 23 |
| Abbildung 11: SG90-Servomotor mit PWM-Ansteuerung (TowerPro SG90 Datasheet, 2021).....                                             | 25 |
| Abbildung 12: RGB-LED mit 4 Pins (Rot, Grün, Blau, gemeinsamer Anschluss) (Kingbright RGB LED Datasheet) .....                     | 26 |
| Abbildung 13: passiver Piezo-Buzzer mit interner Schwingplatte. (Hill, 2015).....                                                  | 27 |
| Abbildung 14: LCD1602-Display mit HD44780-Controller und optionalem I <sup>2</sup> C-Adapter (Ptruzzellis T., 2017).....           | 28 |
| Abbildung 15: Raspberry-Pi-Kameramodul mit CSI-Anschluss (Szeliski, 2022) .....                                                    | 30 |
| Abbildung 16: Vollständiger Schaltplan des Smart-Door-Systems .....                                                                | 32 |
| Abbildung 17: Lokales Webinterface des Smart-Door-Systems .....                                                                    | 37 |
| Abbildung 18: Telegram-Benachrichtigung bei Klingelereignis.....                                                                   | 38 |
| Abbildung 19: Mini-Haus Demonstrationsmodell (Frontansicht).....                                                                   | 41 |
| Abbildung 20: Interne Elektronik und Verdrahtung .....                                                                             | 42 |
| Abbildung 21: System-Log mit Bewegungs- und Helligkeitserkennung .....                                                             | 44 |

## 1. Einleitung

Die zunehmende Digitalisierung des Alltags führt dazu, dass immer mehr technische Systeme in den Bereich des Smart Homes integriert werden. Smart-Home-Systeme ermöglichen die intelligente Vernetzung und Steuerung von Sensoren und Aktoren innerhalb eines Gebäudes und tragen zur Automatisierung sowie zur Erhöhung von Komfort und Sicherheit bei (Stromberg, 2019)

Besonders im Bereich der Sicherheit und Zugangskontrolle gewinnen intelligente Türüberwachungssysteme zunehmend an Bedeutung. Moderne Lösungen erlauben es, Bewegungen zu erkennen, Zugänge elektronisch zu kontrollieren und Benachrichtigungen in Echtzeit zu versenden. Systeme zur Identifikation, beispielsweise auf Basis von RFID-Technologie, werden dabei häufig zur kontaktlosen Authentifizierung eingesetzt (Finkenzeller, 2010)

Klassische mechanische Türsysteme bieten in der Regel keine Möglichkeit zur digitalen Überwachung oder Fernsteuerung. Funktionen wie automatische Reaktionen auf Ereignisse, Statusanzeigen oder Benachrichtigungen bei Besucheraktivität fehlen oftmals vollständig. Gleichzeitig steigt jedoch der Bedarf an komfortablen, sicheren und flexibel erweiterbaren Lösungen im privaten wie auch im halböffentlichen Bereich (Stromberg, 2019)

Ziel dieses Projekts ist die Entwicklung eines intelligenten Smart-Door-Systems auf Basis eines Raspberry Pi. Das System kombiniert verschiedene Sensoren und Aktoren, um eine automatisierte Türüberwachung zu realisieren. Dabei werden Bewegungserkennung, Zugangskontrolle mittels RFID, Tag-/Nacht-Erkennung sowie eine lokale Websteuerung integriert. Zusätzlich wird bei Klingelereignissen eine Benachrichtigung über Telegramm ausgelöst. Ein besonderer Fokus liegt auf der lokalen Verarbeitung der Daten ohne Cloud-Anbindung. Dadurch wird sowohl die Datensicherheit erhöht als auch die Systemunabhängigkeit gewährleistet. Das Projekt wurde als funktionsfähiger Prototyp in einem Mini-Haus-Modell umgesetzt, um eine realitätsnahe Demonstration zu ermöglichen.

Im weiteren Verlauf dieses Berichts werden zunächst die technischen Grundlagen erläutert, anschließend der Entwicklungsprozess beschrieben und schließlich die Hardware- und Softwareimplementierung detailliert dargestellt.

## 1.1 Motivation

Die Motivation für dieses Projekt entstand aus dem zunehmenden Bedarf an intelligenten und automatisierten Sicherheitssystemen im privaten Wohnbereich. Moderne Smart-Home-Technologien ermöglichen es, verschiedene Funktionen wie Beleuchtung, Heizung oder Überwachung digital zu steuern. Besonders im Bereich der Zugangskontrolle bieten intelligente Türsysteme einen erheblichen Mehrwert in Bezug auf Sicherheit, Komfort und Kontrolle.

Klassische mechanische Türsysteme erlauben keine automatische Erkennung von Besuchern oder unbefugten Zugriffsversuchen. Zudem fehlt eine Möglichkeit zur digitalen Statusüberwachung oder zur Fernbenachrichtigung bei Ereignissen wie Bewegung oder Klingeln (Finkenzeller, 2010). Dadurch entsteht eine Sicherheitslücke, insbesondere wenn sich die Bewohner nicht im unmittelbaren Umfeld der Tür befinden.

Ein weiterer Motivationsfaktor war die praktische Umsetzung theoretischer Kenntnisse aus den Bereichen Embedded Systems, Mikrocontrollertechnik und Softwareentwicklung. Ziel war es, Sensorik, Aktorik und Netzwerktechnologien in einem realen Projekt zu kombinieren. Durch die Entwicklung eines funktionierenden Prototyps sollten sowohl Hardware- als auch Softwarekenntnisse vertieft werden. Darüber hinaus sollte das System bewusst ohne Cloud-Anbindung realisiert werden, um die Daten lokal zu verarbeiten und die Abhängigkeit von externen Diensten zu vermeiden. Dies erhöht die Datensicherheit und ermöglicht eine unabhängige Nutzung im lokalen Netzwerk.

## 1.2 Problemstellung

Trotz der zunehmenden Verbreitung von Smart-Home-Technologien bestehen im Bereich der intelligenten Zugangskontrolle weiterhin mehrere technische und sicherheitsrelevante Herausforderungen. Viele handelsübliche Systeme setzen auf cloudbasierte Architekturen, bei denen Sensordaten und Kameraaufnahmen extern gespeichert oder verarbeitet werden. Dies kann zu Datenschutz- und Sicherheitsrisiken führen, insbesondere wenn personenbezogene Daten betroffen sind (ENISA- Datenschutz & IoT, 2021)

Darüber hinaus weisen zahlreiche IoT-Geräte bekannte Sicherheitslücken auf, beispielsweise unzureichende Zugriffskontrollen, fehlende Verschlüsselung oder unsichere Standardkonfigurationen (National Institute of Standards and Technology (NIST), 2019). Insbesondere vernetzte Überwachungssysteme sind potenzielle Angriffsziele, da sie sensible Informationen über Zutrittsverhalten und Anwesenheit liefern.

Ein weiteres Problem besteht in der mangelnden Modularität vieler kommerzieller Systeme. Erweiterungen oder individuelle Anpassungen sind häufig nur eingeschränkt möglich. Zudem sind viele Lösungen proprietär aufgebaut und erlauben keine transparente Systemarchitektur oder eigene Softwareanpassung.

Vor diesem Hintergrund ergibt sich die zentrale Problemstellung dieses Projekts:

Wie kann ein intelligentes Türüberwachungssystem realisiert werden, das

- lokal betrieben wird,
- datenschutzkonform arbeitet,
- modular erweiterbar ist,
- mehrere Sicherheitsmechanismen kombiniert
- und dennoch mit kostengünstiger Hardware umgesetzt werden kann?

Ziel ist es, eine eigenständige, lokal kontrollierte Systemarchitektur zu entwickeln, die typische Schwachstellen cloudbasierter oder proprietärer Systeme vermeidet und gleichzeitig praxisnahe Sicherheitsfunktionen bereitstellt.

### 1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Konzeption und Realisierung eines intelligenten Smart-Door-Systems auf Basis eines Raspberry Pi. Das System soll verschiedene Sensoren und Aktoren miteinander kombinieren, um eine automatisierte und sichere Türüberwachung zu ermöglichen.

Im Mittelpunkt steht die Entwicklung eines funktionsfähigen Prototyps, der folgende Anforderungen erfüllt:

- Erkennung von Bewegungen im Eingangsbereich mittels PIR-Sensors
- Unterscheidung zwischen Tag- und Nachtbetrieb durch einen LDR-Sensor
- Zugangskontrolle über RFID-Karten
- Automatische Türöffnung und -schließung über einen Servomotor
- Akustische und visuelle Statusanzeige durch Buzzer und RGB-LED
- Lokale Statusanzeige über ein LCD-Display
- Bereitstellung einer webbasierten Benutzeroberfläche im lokalen Netzwerk

Versand einer Telegramm-Benachrichtigung bei Klingelereignissen

Ein wesentliches Ziel besteht darin, sämtliche Daten lokal zu verarbeiten, ohne auf Cloud-Dienste angewiesen zu sein. Dadurch soll ein datenschutzfreundliches und unabhängiges System realisiert werden.

Abschließend soll das Projekt als vollständig integrierter Demonstrationsprototyp in Form eines Mini-Hauses präsentiert werden, um die Funktionalität anschaulich darzustellen und praxisnah zu validieren.

## 2. Grundlagen und technische Konzepte

In diesem Kapitel werden die technischen Grundlagen erläutert, die für die Entwicklung des Smart-Door-Systems relevant sind. Dazu gehören Konzepte aus dem Bereich Smart Home, Embedded Systems, Kommunikationsprotokolle sowie Sensor und Aktor Technik. Diese theoretischen Grundlagen bilden die Basis für das Verständnis der später beschriebenen Hardware- und Softwareimplementierung.

Moderne Smart-Home-Systeme basieren auf der Vernetzung verschiedener Sensoren und Aktoren, die durch eine zentrale Steuereinheit koordiniert werden. Dabei kommen unterschiedliche Kommunikationsprotokolle sowie digitale und analoge Signalverarbeitung zum Einsatz. Im vorliegenden Projekt übernimmt der Raspberry Pi diese zentrale Steuerfunktion.

Im Folgenden werden zunächst grundlegende Konzepte intelligenter Hausautomatisierung beschrieben, bevor anschließend die eingesetzten technischen Schnittstellen und Bauelemente näher erläutert werden.

### 2.1 Smart-Home-Systeme

Smart-Home-Systeme bezeichnen die intelligente Vernetzung und Automatisierung technischer Geräte innerhalb eines Wohngebäudes. Ziel solcher Systeme ist es, Komfort, Sicherheit, Energieeffizienz und Benutzerfreundlichkeit zu erhöhen. Dabei werden Sensoren, Aktoren und zentrale Steuereinheiten miteinander verbunden, um automatisch auf bestimmte Ereignisse reagieren zu können (Stromberg, 2019)

Ein typisches Smart-Home-System besteht aus drei zentralen Komponenten:

1. Sensoren, die physikalische Größen erfassen (z. B. Bewegung, Licht, Temperatur)
2. Aktoren, die mechanische oder elektrische Aktionen ausführen (z. B. Tür öffnen, Licht einschalten)
3. Steuereinheiten, die Sensordaten verarbeiten und entsprechende Aktionen auslösen

Diese Struktur entspricht dem Grundprinzip cyber-physischer Systeme, bei denen physische Prozesse digital überwacht und gesteuert werden (Lee, 2008)

Die Kommunikation zwischen den Komponenten erfolgt über digitale Schnittstellen oder drahtlose Netzwerke. Häufig eingesetzte Technologien sind WLAN, Bluetooth oder ZigBee sowie kabelgebundene Protokolle wie I<sup>2</sup>C und SPI, die insbesondere in Embedded-Systemen Anwendung finden (Raspberry pi Foundation, 2023).

## Smart Door System

Ein wesentliches Merkmal moderner Smart-Home-Systeme ist die ereignisbasierte Logik. Dabei wird nicht dauerhaft manuell gesteuert, sondern das System reagiert automatisch auf bestimmte Zustände. Beispielsweise kann bei erkannter Bewegung automatisch eine Beleuchtung aktiviert oder eine Benachrichtigung versendet werden. Dieses Prinzip ist ein zentrales Element des Internet of Things (IoT), bei den vernetzten Geräten autonom miteinander interagieren (Atzori, Lera, & Morabito, 2010).

Im Bereich der Türüberwachung bieten Smart-Home-Lösungen Funktionen wie:

- Automatische Bewegungserkennung
- Zugangskontrolle per Chipkarte oder Smartphone
- Videoüberwachung
- Echtzeit-Benachrichtigungen
- Fernsteuerung über Web- oder Mobilanwendungen

Im vorliegenden Projekt wird ein solches Smart-Home-Prinzip auf ein Türsystem angewendet. Die Steuerung erfolgt bewusst lokal, um Datenschutz und Systemunabhängigkeit zu gewährleisten. Lokale Architekturen reduzieren potenzielle Sicherheitsrisiken cloudbasierter Lösungen (National Institute of Standards and Technology (NIST), 2019)

## 2.2 Raspberry Pi als Embedded System

Der Raspberry Pi ist ein Einplatinencomputer (Single Board Computer), der häufig im Bereich der Embedded Systems eingesetzt wird. Ein Embedded System ist ein spezialisiertes Computersystem, das in ein technisches Gesamtsystem integriert ist und dort bestimmte Steuerungs- oder Regelungsaufgaben übernimmt. Im Gegensatz zu klassischen Desktop-Computern ist ein Embedded System meist auf eine klar definierte Funktion ausgerichtet. Es arbeitet energieeffizient, kompakt und wird häufig in industriellen oder hausautomatisierten Anwendungen verwendet (Michael Barr, 2006).

Der Raspberry Pi kombiniert die Eigenschaften eines vollwertigen Computers mit der Flexibilität eines Mikrocontrollers. Er verfügt über:

- Einen leistungsfähigen ARM-Prozessor
- Arbeitsspeicher (RAM)
- USB- und Netzwerkanschlüsse
- Digitale Ein- und Ausgänge (GPIO)
- Schnittstellen wie I<sup>2</sup>C, SPI und UART
- Einen dedizierten Kameraanschluss (CSI)

Diese Eigenschaften sind in der offiziellen technischen Dokumentation der Raspberry Pi Foundation beschrieben (Raspberry pi Foundation, 2023).

Im vorliegenden Projekt übernimmt der Raspberry Pi die zentrale Steuerfunktion des Smart-Door-Systems. Er verarbeitet die Sensordaten, trifft logische Entscheidungen und steuert die Aktoren entsprechend an. Zusätzlich stellt er eine lokale Weboberfläche bereit und ermöglicht die Kommunikation mit einem Telegram-Bot. Ein wesentlicher Vorteil des Raspberry Pi gegenüber Mikrocontrollern wie dem ESP32 liegt in der höheren Rechenleistung und der Möglichkeit, ein vollständiges Linux-Betriebssystem auszuführen. Dadurch können komplexe Softwarestrukturen, Multithreading und Netzwerkdienste problemlos realisiert werden (Raspberry pi Foundation, 2023).

## 2.3 GPIO-Schnittstellen

GPIO steht für *General Purpose Input/Output* und bezeichnet frei programmierbare Ein- und Ausgänge eines Mikrocontrollers oder Einplatinencomputers wie dem Raspberry Pi. Diese Pins ermöglichen die direkte Kommunikation zwischen der Software und der realen Hardwarewelt. Über GPIO-Schnittstellen können externe Sensoren ausgelesen und Aktoren angesteuert werden (Raspberry pi Foundation, 2023)

GPIO-Pins können in zwei grundlegenden Modi betrieben werden:

- Input (Eingang): Der Pin liest ein Signal, beispielsweise von einem Sensor.
- Output (Ausgang): Der Pin gibt ein Signal aus, beispielsweise zum Ansteuern einer LED oder eines Servomotors.

Der Raspberry Pi arbeitet mit einer Logikspannung von 3,3 Volt, weshalb angeschlossene Komponenten ebenfalls mit dieser Spannung kompatibel sein müssen. Eine direkte Verbindung mit 5-Volt-Signalen kann den Raspberry Pi beschädigen (Raspberry pi Foundation, 2023).

Im Smart-Door-System werden die GPIO-Schnittstellen für mehrere Funktionen eingesetzt:

- Der PIR-Sensor liefert ein digitales HIGH-/LOW-Signal an einen GPIO-Eingang.
- Der Klingeltaster wird als digitaler Eingang mit internem Pull-Up-Widerstand betrieben.
- Der Servomotor wird über einen GPIO-Ausgang mittels PWM (Pulsweitenmodulation) angesteuert.
- Die RGB-LED sowie der Buzzer werden über digitale Ausgangspins geschaltet.

PWM ist ein Verfahren zur Steuerung analog wirkender Ausgangsgrößen durch Variation der Pulsbreite eines digitalen Signals (Michael Barr, 2006)

Ein wichtiger Aspekt bei der Nutzung von GPIO-Pins ist die Signalstabilität. Mechanische Taster erzeugen beim Drücken sogenannte Prell-Effekte, weshalb in der Software eine Entprellung implementiert werden muss (Hill, 2015). Ebenso ist auf eine saubere Masseverbindung (GND) zwischen allen Komponenten zu achten, um Signalstörungen zu vermeiden.

## 2.4 Kommunikationsprotokolle (I<sup>2</sup>C und SPI)

In modernen Embedded-Systemen werden verschiedene Kommunikationsprotokolle eingesetzt, um Daten zwischen der zentralen Steuereinheit und externen Modulen auszutauschen. Im vorliegenden Smart-Door-System kommen insbesondere die seriellen Bussysteme I<sup>2</sup>C und SPI zum Einsatz. Beide Protokolle sind in eingebetteten Systemen weit verbreitet und werden häufig zur Anbindung von Sensoren, Displays oder Identifikationsmodulen verwendet (Raspberry pi Foundation, 2023).

I<sup>2</sup>C (Inter-Integrated Circuit) ist ein serielles Zwei-Draht-Kommunikationsprotokoll, das für die Verbindung mehrerer Geräte über nur zwei Leitungen entwickelt wurde (NXP Semiconductors, 2021). Die beiden Leitungen sind:

- SDA (Serial Data) – Datenleitung
- SCL (Serial Clock) – Taktleitung

Ein Master-Gerät (hier der Raspberry Pi) steuert die Kommunikation, während mehrere Slave-Geräte über individuelle Adressen angesprochen werden können. Die Adressierung erfolgt typischerweise über 7-Bit- oder 10-Bit-Adressen. Ein wesentlicher Vorteil von I<sup>2</sup>C besteht darin, dass mehrere Geräte gleichzeitig an denselben Bus angeschlossen werden können. Dies reduziert den Verdrahtungsaufwand erheblich.

Im Smart-Door-System wird I<sup>2</sup>C verwendet für:

- das LCD1602-Display (über ein PCF8574-I<sup>2</sup>C-Modul)
- den ADS7830 Analog-Digital-Wandler zur Auswertung des LDR-Sensors

Die Geräte werden über ihre jeweilige I<sup>2</sup>C-Adresse identifiziert, beispielsweise 0x27 für das LCD oder 0x4B für den ADC.

SPI (Serial Peripherie Interface) ist ebenfalls ein serielles Kommunikationsprotokoll, das jedoch höhere Datenübertragungsraten als I<sup>2</sup>C ermöglicht. SPI wurde ursprünglich von Motorola entwickelt und wird insbesondere dann eingesetzt, wenn schnelle und zuverlässige Datenübertragungen erforderlich sind (Motorola, 2003).

SPI benötigt in der Regel vier Leitungen:

- MOSI (Master Out Slave In) – Daten vom Master zum Slaven
- MISO (Master In Slave Out) – Daten vom Slaven zum Master
- SCK (Serial Clock) – Taktsignal
- CS (Chip Select) – Auswahl des jeweiligen Slave-Geräts

Im Gegensatz zu I<sup>2</sup>C besitzt SPI keine Adressierung über den Bus, sondern verwendet für jedes Slave-Gerät eine separate Chip-Select-Leitung.

Im Smart-Door-System wird SPI zur Kommunikation mit dem RFID-Modul MFRC522 eingesetzt. Die schnelle Datenübertragung ermöglicht eine zuverlässige und verzögerungsfreie Identifikation von RFID-Karten.

## 2.5 Sensor- und Aktorprinzipien

In automatisierten technischen Systemen übernehmen Sensoren und Aktoren zentrale Funktionen. Sensoren dienen der Erfassung physikalischer Größen wie Bewegung, Licht, Temperatur oder Druck und wandeln diese in elektrische Signale um, die von einem Steuerungssystem weiterverarbeitet werden können (Hill, 2015). Dieser Vorgang wird als Signalumwandlung oder Transduktion bezeichnet.

Aktoren hingegen setzen elektrische Signale in physikalische Wirkungen um. Dabei kann es sich um mechanische Bewegungen, optische Signale oder akustische Ausgaben handeln. Typische Beispiele sind Elektromotoren, Servomotoren, LEDs oder akustische Signalgeber (Buzzer). Aktoren stellen somit die Schnittstelle zwischen digitaler Steuerlogik und physischer Umwelt dar (Michael Barr, 2006).

### 3.Entwicklungsverlauf

Die Entwicklung des Smart-Door-Systems erfolgte in mehreren Phasen. Ziel war es, schrittweise von einer einfachen, kompakten Lösung zu einer stabilen und erweiterbaren Systemarchitektur zu gelangen. Dabei wurden sowohl Hardware- als auch Softwareaspekte kontinuierlich getestet, bewertet und angepasst. Zu Beginn lag der Fokus auf einer möglichst kostengünstigen und kompakten Umsetzung mit integrierter Kamera und WLAN-Funktionalität. Im weiteren Verlauf zeigte sich jedoch, dass steigende Anforderungen an Rechenleistung, Schnittstellen und Softwarestruktur einen Plattformwechsel erforderlich machten.

#### 3.1 Erste Phase: ESP32-CAM

In der ersten Entwicklungsphase wurde das System mit einem ESP32-CAM-Modul ( Abbildung 1) umgesetzt. Der ESP32 ist ein Mikrocontroller mit integrierter WLAN-Funktionalität, der sich besonders für IoT-Anwendungen eignet (ESP32, 2023)

Das Modul kombiniert:

- Microcontroller (Xtensa Dual-Core)
- WLAN-Schnittstelle
- Kamera (OV2640)
- Digitale GPIO-Pins

Der ESP32-CAM eignete sich besonders gut für erste Funktionstests, da grundlegende Features wie Bewegungserkennung, Fotoaufnahme und drahtlose Datenübertragung schnell umgesetzt werden konnten. Erste Tests zeigten, dass Bilder erfolgreich aufgenommen und über das Netzwerk übertragen werden konnten.

Allerdings traten im Verlauf der Entwicklung mehrere Einschränkungen auf. Die begrenzte Anzahl nutzbarer GPIO-Pins erschwerte die gleichzeitige Anbindung mehrerer Sensoren und Aktoren. Zudem erwies sich die Softwareentwicklung bei wachsender Komplexität als unübersichtlich und fehleranfällig. Auch die parallele Nutzung von Kamera, WLAN und weiteren Modulen führte teilweise zu Stabilitätsproblemen.

Diese Erkenntnisse führten schließlich zur Entscheidung, das Projekt auf eine leistungsfähigere Plattform zu migrieren.



Abbildung 1: ESP32-CAM (ESP32, 2023)

## 3.2 Umstieg auf Raspberry Pi

Aufgrund der genannten Einschränkungen wurde das System in einer zweiten Phase auf einen Raspberry Pi portiert. Der Raspberry Pi (Abbildung 2) bietet ein vollständiges Linux-Betriebssystem, deutlich höhere Rechenleistung sowie umfangreiche Schnittstellen wie GPIO, I<sup>2</sup>C, SPI, PWM und dedizierte Kameraanschlüsse. Dadurch ermöglicht er Multithreading, Netzwerkdienste und komplexe Softwarearchitekturen (Raspberry pi Foundation, 2023).

Durch den Umstieg konnten komplexere Softwarestrukturen umgesetzt werden, beispielsweise ein lokaler Webserver, eine strukturierte Ereignislogik sowie die Integration zusätzlicher Kommunikationsdienste. Auch die Nutzung der Raspberry-Pi-Kamera erwies sich als stabiler und leistungsfähiger im Vergleich zur ESP32-CAM.

Der Plattformwechsel stellte einen entscheidenden Schritt dar, um das Projekt in Richtung eines robusten und praxisnahen Smart-Door-Systems weiterzuentwickeln.

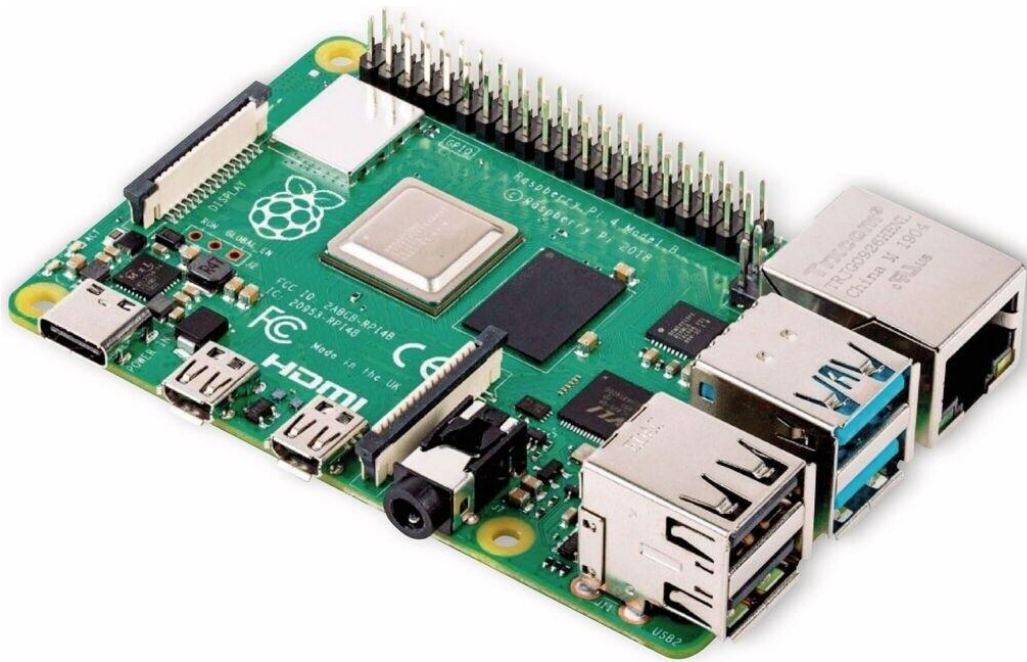


Abbildung 2: Raspberry Pi 4 Model B (Raspberry pi Foundation, 2023)

## 4. Hardwarekonzept und Komponenten

Dieses Kapitel beschreibt die eingesetzten Hardwarekomponenten sowie deren jeweilige Funktion innerhalb des Systems. Das Hardwarekonzept basiert auf einer modularen Architektur, bei der alle Sensoren und Aktoren zentral über den Raspberry Pi gesteuert werden.

### 4.1 Systemübersicht

Das Smart-Door-System besteht aus einer zentralen Steuereinheit (Raspberry Pi), mehreren Sensoren zur Umwelterfassung sowie verschiedenen Aktoren zur Umsetzung physischer Reaktionen. Diese sind in der Abbildung 3 dargestellt.

Das System arbeitet nachfolgendem Prinzip:

1. Sensor erkennt ein Ereignis
2. Raspberry Pi verarbeitet die Daten
3. Logische Entscheidung wird getroffen
4. Aktor wird angesteuert
5. Optional: Benachrichtigung oder Web-Update

Diese Architektur ermöglicht eine klare Trennung zwischen Datenerfassung, Verarbeitung und Reaktion.

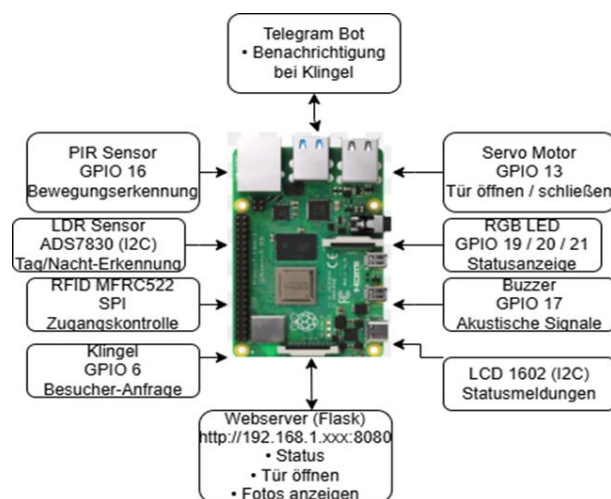


Abbildung 3: Systemarchitektur des Smart-Door-Systems mit allen Komponenten

## 4.2 Raspberry Pi

Der Raspberry Pi im (Abbildung 2) fungiert als zentrale Steuereinheit des Systems. Er verarbeitet Sensordaten, führt die Entscheidungslogik aus und steuert die Aktoren an.

Zusätzlich übernimmt er:

- Betrieb des lokalen Webserver
- Speicherung von Kamerabildern
- Kommunikation mit dem Telegram-Bot
- Verwaltung der GPIO-Schnittstellen

Durch das Linux-Betriebssystem können Prozesse parallel ausgeführt werden, wodurch eine stabile Echtzeitreaktion ermöglicht wird.

## 4.3 PIR-Bewegungssensor

Ein PIR-Sensor (*Passive Infrared Sensor*) ist ein passiver Infrarotsensor, der Änderungen der Wärmestrahlung im Infrarotbereich detektiert. Jeder menschliche Körper emittiert aufgrund seiner Temperatur ( $\sim 37^\circ\text{C}$ ) elektromagnetische Strahlung im Infrarotspektrum (Hill, 2015).

Der PIR-Sensor (Abbildung 5) enthält ein pyroelektrisches Element, das auf Änderungen der einfallenden Infrarotstrahlung reagiert. Der Sensor misst keine absolute Temperatur, der erkennt nur Änderungen der Wärmestrahlung. Bewegt sich eine Person durch das Erfassungsfeld, verändert sich die gemessene Infrarotintensität zwischen zwei Sensorelementen. Diese Differenz erzeugt ein elektrisches Signal, das als HIGH- oder LOW-Ausgang ausgegeben wird.

-Technische Eigenschaften (typisch für HC-SR501):

- Betriebsspannung: 3,3–5 V
- Stromaufnahme: ca. 50–70  $\mu\text{A}$  (sehr energieeffizient)
- Reichweite: 5–7 Meter
- Erfassungswinkel: ca.  $120^\circ$
- Digitaler Ausgang (HIGH = Bewegung erkannt)

## Smart Door System

Neben dem eingesetzten PIR-Sensor existieren weitere Technologien zur Bewegungserkennung, die je nach Anwendung spezifische Vor- und Nachteile bieten. Die Auswahl eines geeigneten Sensors hängt dabei von Faktoren wie Reichweite, Energieverbrauch, Umgebungsbedingungen, Datenschutzanforderungen und Kosten ab. Die Abbildung 4 zeigt ein Vergleich von einigen.

| Technologie | Reichweite | Stromverbrauch | Genauigkeit | Preis   | Datenschutz |
|-------------|------------|----------------|-------------|---------|-------------|
| PIR         | 5–7 m      | sehr gering    | mittel      | niedrig | sehr gut    |
| Ultraschall | 2–4 m      | mittel         | hoch        | niedrig | gut         |
| Radar       | 5–10 m     | mittel         | hoch        | mittel  | gut         |
| Kamera      | >10 m      | hoch           | sehr hoch   | hoch    | kritisch    |

Abbildung 4: Vergleich der Technologien



Abbildung 5: PIR-Sensor (Hill, 2015)

## 4.4 LDR mit ADC (ADS7830)

### 4.4.1 Funktionsprinzip des LDR

Ein LDR (Light Dependent Resistor) ist ein lichtabhängiger Widerstand, dessen elektrischer Widerstand sich in Abhängigkeit von der einfallenden Lichtintensität ändert. Physikalisch basiert er auf dem photoelektrischen Effekt in Halbleitermaterialien. Trifft Licht auf das Halbleitermaterial (z. B. Cadmiumsulfid,

CdS), werden zusätzliche Ladungsträger erzeugt. Dadurch sinkt der Widerstand bei zunehmender Beleuchtung (Hill, 2015).

Typisches Verhalten:

- ☐ Dunkelheit → hoher Widerstand ( $k\Omega$ – $M\Omega$ )
- ☐ Helligkeit → niedriger Widerstand (wenige  $k\Omega$ )

Der LDR in Abbildung 7 liefert jedoch kein digitales Signal, sondern eine analoge Spannungsänderung und der Raspberry Pi 4 Model B besitzt keine integrierten analogen Eingänge. Deshalb wird ein Analog-Digital-Wandler (ADC) benötigt.

Es wird in diesem Projekt der ADS7830 eingesetzt. Der ADS7830 in Abbildung 6 ist ein:

- ☐ 8-Bit Analog-Digital-Wandler
- ☐ mit I<sup>2</sup>C-Schnittstelle
- ☐ bis zu 8 analoge Eingangskanäle
- ☐ Betriebsspannung: 2,7–5,5 V

Er wandelt die analoge Spannung des LDR-Spannungsteilers in einen digitalen Wert (0–255) um, der vom Raspberry Pi verarbeitet werden kann (Texas Instruments (ADS7830 Datasheet), 2023).

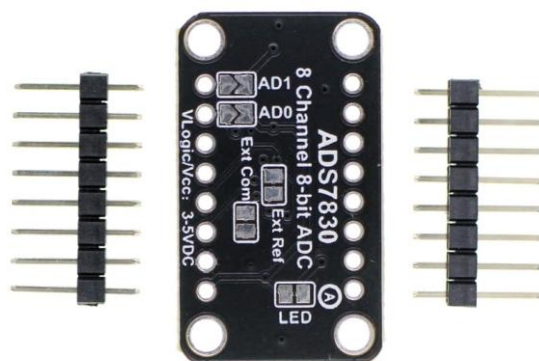


Abbildung 6: ADS7830 Analog-Digital-Wandler mit I<sup>2</sup>C-Schnittstelle (Texas Instruments (ADS7830 Datasheet), 2023)

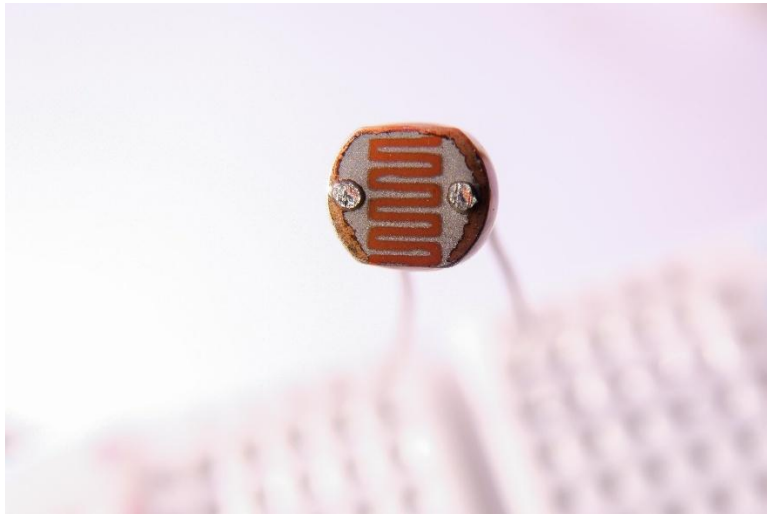


Abbildung 7: LDR (lichtabhängiger Widerstand) (Hill, 2015)

Neben dem eingesetzten LDR-Sensor gibt alternative Helligkeitssensoren die digitalen Signale liefern wie:

Digitale Lux-Sensoren (z. B. BH1750)

- Direkt digitaler Lux-Wert
- I<sup>2</sup>C-Schnittstelle
- Höhere Genauigkeit als LDR
- Kalibrierte Messwerte geeignet

(ROHM (BH1750 Datasheet), 2026)  
2023)

TSL2561 (Lichtsensor)

Misst sichtbares + IR-Licht  
I<sup>2</sup>C  
Bessere Messgenauigkeit  
Für präzise Lichtmessungen

(AMS (TSL2561 Datasheet),

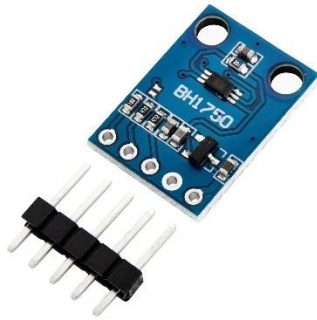


Abbildung 9: BH1750 digitaler Lux-Sensor mit I<sup>2</sup>C-Schnittstelle

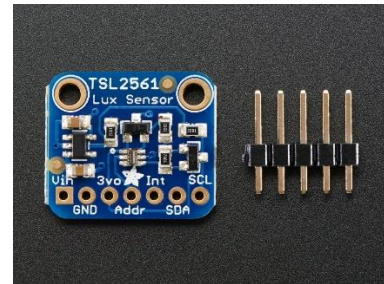


Abbildung 8: TSL2561 digitaler Lichtsensor

Der BH1750 ist ein digitaler Umgebungslichtsensor mit integrierter Analog-Digital-Wandlung. Er gibt direkt kalibrierte Lux-Werte über die I<sup>2</sup>C-Schnittstelle aus und benötigt keinen externen ADC (ROHM (BH1750 Datasheet), 2026).

Der TSL2561 ist ein digitaler Lichtsensor mit zwei getrennten Fotodiodenkanälen (sichtbares Licht und Infrarot). Dadurch ermöglicht er eine genauere Bestimmung der realen Beleuchtungsstärke. Die Kommunikation erfolgt ebenfalls über I<sup>2</sup>C (AMS (TSL2561 Datasheet), 2023). In der unteren Tabelle 1 ist ein Vergleich von der Unterschiedlichen Helligkeitssensoren dargestellt.

| Sensor  | Genauigkeit | ADC nötig | Kosten      | Komplexität  |
|---------|-------------|-----------|-------------|--------------|
| LDR     | Mittel      | Ja        | Sehr gering | Sehr einfach |
| BH1750  | Hoch        | Nein      | Mittel      | Einfach      |
| TSL2561 | Sehr hoch   | Nein      | Mittel      | Mittel       |

Tabelle 1: Vergleich Helligkeitssensoren

## 4.5 RFID-Modul (MFRC522)

### 4.5.1 Grundprinzip der RFID-Technologie

RFID steht für Radio Frequency Identification und bezeichnet ein Verfahren zur kontaktlosen Identifikation von Objekten mittels elektromagnetischer Wellen. Ein RFID-System besteht grundsätzlich aus zwei Komponenten, der RFID-Reader (Lesegerät) und der RFID-Tag (Transponder, z. B. Karte oder Chip).

Der MFRC522 im Abbildung 10 ist ein weit verbreitetes RFID-Lesemodul, das im 13,56 MHz HF-Bereich arbeitet. Es kommuniziert typischerweise mit sogenannten MIFARE-Tags nach dem ISO/IEC 14443-A Standard (Finkenzeller, 2010).

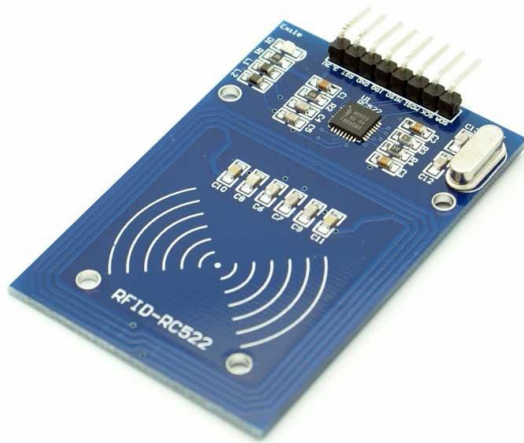


Abbildung 10: MFRC522 RFID-Modul mit SPI-Schnittstelle

### 4.5.2 Funktionsweise des MFRC522

Das Modul erzeugt ein elektromagnetisches Feld mit 13,56 MHz. Befindet sich eine RFID-Karte im Nahbereich (typisch 2–5 cm), wird im Transponder durch induktive Kopplung Energie erzeugt. Der Tag benötigt keine eigene Batterie (passiver RFID-Tag).

Der Raspberry Pi kommuniziert mit dem MFRC522 über das SPI-Protokoll. (NXP Semiconductors (MFRC522 Datasheet), 2023) und Der Datenaustausch erfolgt über Load Modulation:

- Reader sendet Abfrage
- Tag moduliert das empfangene Feld
- Reader interpretiert die Antwort

Das Modul ist besonders geeignet für Zugangskontrollsysteme mit kurzer Distanz, da es eine zuverlässige Identifikation bei direktem Annähern ermöglicht. Der eingesetzte MFRC522 Modul verfügt über folgende technische Eigenschaften:

## Smart Door System

- Betriebsfrequenz: 13,56 MHz
- Betriebsspannung: 3,3 V
- Kommunikation: SPI
- Lesedistanz: ca. 2–5 cm
- Unterstützt: MIFARE 1K, 4K

Im vorliegenden Projekt übernimmt das RFID-Modul eine zentrale Funktion innerhalb des Zugangskontrollsystems. Zunächst dient es der eindeutigen Identifikation von RFID-Tags anhand ihrer individuellen UID (Unique Identifier). Jede RFID-Karte besitzt eine fest zugewiesene Identifikationsnummer, die vom Lesemodul ausgelesen wird.

Nach dem Auslesen der UID erfolgt ein automatischer Abgleich mit einer im System hinterlegten Liste autorisierter Identifikationsnummern. Ist die UID in dieser Liste enthalten, wird der Zugang freigegeben und die Tür durch Ansteuerung des Servomotors geöffnet. Erfolgt keine Übereinstimmung, bleibt die Tür geschlossen. Zusätzlich wird jedes Identifikationsereignis im lokalen Webinterface protokolliert. Dadurch kann nachvollzogen werden, wann und mit welcher UID ein Zugriffsversuch stattgefunden hat. Dies erhöht die Transparenz und Sicherheit des Systems. Ein wesentlicher Vorteil dieser Implementierung liegt in der schnellen und kontaktlosen Authentifizierung. Im Gegensatz zu mechanischen Schlüsselsystemen entfällt der physische Verschleiß, und verlorene oder kompromittierte Karten können einfach durch Entfernen der entsprechenden UID aus der Autorisierungsliste deaktiviert werden.

## 4.6 Servomotor

Ein Servomotor (Abbildung 11) ist ein elektromechanischer Aktor, der zur präzisen Positionssteuerung von rotierenden Elementen eingesetzt wird. Im Gegensatz zu herkömmlichen Gleichstrommotoren besitzt ein Servomotor eine integrierte Regelungseinheit, die eine definierte Winkelposition ansteuert und stabil hält (Bolton W, 2015).

Ein typischer Modellbau-Servomotor besteht aus:

- einem Gleichstrommotor
- einem Getriebe zur Drehmomentverstärkung
- einem Potentiometer zur Positionsrückmeldung
- einer internen Steuerelektronik

Die interne Elektronik vergleicht die gewünschte Position mit der aktuellen Position und regelt den Motor entsprechend nach. Dadurch entsteht ein geschlossenes Regelkreissystem. Servomotoren werden üblicherweise über PWM-Signale

gesteuert. Der Raspberry Pi erzeugt dieses PWM-Signal über einen GPIO-Pin (TowerPro SG90 Datasheet, 2021).

Die Pulsbreite bestimmt direkt die gewünschte Winkelposition.

Typische Parameter:

- Frequenz: 50 Hz
- Betriebsspannung: 4,8 – 6 V
- Drehmoment: ca. 1,8 kg·cm
- Stromaufnahme: ca. 100–500 mA (abhängig von Last)
- Steuerung: PWM
- Drehbereich: ca. 180°
- Pulsbreite:
  - ca. 1 ms → 0°
  - ca. 1,5 ms → 90°
  - ca. 2 ms → 180°

Im vorliegenden Projekt übernimmt der Servomotor die mechanische Öffnung und Schließung der Tür. Bei erfolgreicher Authentifizierung wird der Servo auf eine definierte Winkelposition (z. B. 90°) bewegt, wodurch die Tür entriegelt wird. Nach Ablauf einer festgelegten Zeit wird der Servo zurück in die Ausgangsposition gefahren.

Die Wahl eines Servomotors ermöglicht eine kompakte und kostengünstige Realisierung der mechanischen Bewegung ohne komplexe Getriebe- oder Motortreiber-Schaltungen.



Abbildung 11: SG90-Servomotor mit PWM-Ansteuerung (TowerPro SG90 Datasheet,

2021)

## 4.7 RGB-LED

Eine RGB-LED in Abbildung 12 ist eine Leuchtdiode, die drei einzelne Halbleiter-LEDs in einem Gehäuse kombiniert (Rot, Grün, Blau). Durch Kombination dieser drei Grundfarben können nahezu alle sichtbaren Farben erzeugt werden. Dieses Prinzip basiert auf der additiven Farbmischung des RGB-Farbmodells (Wyszeck G. & Stiles W., 2000).

Die Lichtemission entsteht durch Elektrolumineszenz: Wird eine LED in Durchlassrichtung betrieben, rekombinieren Elektronen und Defektelektronen im Halbleitermaterial, wobei Photonen emittiert werden.

RGB-Leds gibt es in zwei Varianten: Common Cathode (gemeinsame Kathode), hier sind alle Minusanschlüsse verbunden und Common Anode (gemeinsame Anode) hier sind alle Plusanschlüsse verbunden. Im Smart-Door-System wird eine RGB-LED verwendet, die über drei separate GPIO-Pins angesteuert wird. Jede Farbe wird einzeln geschaltet und übernimmt eine visuelle Statusanzeige (Grün für Zugang autorisiert, Rot für Zugriff verweigert, Weiß für Nachtbeleuchtung und Blau für klingelt).

Die elektrischen Eigenschaften und PWM-Ansteuerung der RGB-LED sind:

- Betriebsspannung:
  - Rot: ca. 1,8–2,2 V
  - Grün/Blau: ca. 2,8–3,3 V
- Strom: ca. 10–20 mA pro Farbe
- Erforderlicher Vorwiderstand: z. B. 220  $\Omega$  – 330  $\Omega$
- 100 % Rot + 0 % Grün + 0 % Blau → Rot
- 100 % Rot + 100 % Grün → Gelb
- 100 % Rot + 100 % Grün + 100 % Blau → Weiß

Die Helligkeit jeder Farbe wird durch die Pulsbreite bestimmt (Kingbright RGB LED Datasheet).

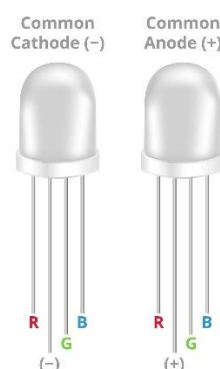


Abbildung 12: RGB-LED mit 4 Pins (Rot, Grün, Blau, gemeinsamer Anschluss)  
(Kingbright RGB LED Datasheet)

## 4.8 Buzzer

Ein Buzzer im Abbildung 13 ist ein elektroakustischer Aktor, der elektrische Signale in hörbare Schallwellen umwandelt. Er dient zur akustischen Signalisierung von Zuständen oder Ereignissen. Man unterscheidet grundsätzlich zwischen Aktiver Buzzer und passiver Buzzer (Hill, 2015).

Ein aktiver Buzzer enthält eine integrierte Oszillatorschaltung. Der Buzzer erzeugt automatisch einen festen Ton, wenn eine Gleichspannung angelegt wird. Der hat keine mögliche Tonhöhenvariation, sehr einfache Ansteuerung (High = Ton) und kein PWM-Signal notwendig. Im gegensagt zu aktiven Buzzer enthält den passiven Buzzer keinen Oszillator. Seine Tonerzeugung erfolgt durch ein externes Wechsel- oder PWM-Signal und so kann die Frequenz gezielt verändert werden (Hill, 2015).



Abbildung 13: passiver Piezo-Buzzer mit interner Schwingplatte. (Hill, 2015)

### Technische Eigenschaften

- Betriebsspannung: 3,3–5 V
- Stromaufnahme: 20–40 mA
- Frequenzbereich (passiv): 1–5 kHz
- Schalldruck: ca. 85–95 dB

## 4.9 LCD1602-Display

Das LCD1602 in Abbildung 14 ist ein alphanumerisches Flüssigkristalldisplay mit zwei Zeilen und jeweils 16 Zeichen pro Zeile. Es basiert auf der LCD-Technologie (Liquid Crystal Display), bei der Flüssigkristalle durch elektrische Spannung ihre optischen Eigenschaften verändern.

Ein LCD emittiert kein eigenes Licht, sondern moduliert das Hintergrundlicht (Backlight). Die Flüssigkristalle drehen die Polarisationssebene des Lichts abhängig von der angelegten Spannung. Dadurch erscheinen Zeichen dunkel auf hellem Hintergrund (Ptruzzellis T., 2017).



Abbildung 14: LCD1602-Display mit HD44780-Controller und optionalem I<sup>2</sup>C-Adapter (Ptruzzellis T., 2017)

### Aufbau des LCD1602

Ein typisches LCD1602-Modul besteht aus:

- LCD-Controller (meist HD44780-kompatibel)
- 2 × 16 Zeichenmatrix
- Hintergrundbeleuchtung (LED)
- Daten- und Steuerepins

## Smart Door System

Der HD44780-Controller ist ein weit verbreiteter Standard für alphanumerische Displays (Hitachi (HD44780U Datasheet)).

### Ansteuerung im Projekt

Im Smart-Door-System wird das LCD1602 über ein I<sup>2</sup>C-Interface-Modul (PCF8574) angesteuert. Dadurch reduziert sich die Anzahl der benötigten GPIO-Pins erheblich.

Ohne I<sup>2</sup>C:

- 6–8 GPIO-Pins erforderlich

Mit I<sup>2</sup>C:

- Nur 2 Leitungen (SDA, SCL)

Das Display dient zur Anzeige von:

- Systemstatus (Bereit, Bewegung erkannt)
- RFID-Ergebnissen (Zugang erlaubt/verweigert)
- Klingelereignissen
- Debug-Informationen
- 

### Technische Eigenschaften

- Anzeige: 16 × 2 Zeichen
- Betriebsspannung: 5 V
- Logikpegel: 5 V (I<sup>2</sup>C-Modul meist kompatibel)
- Schnittstelle: Parallel oder I<sup>2</sup>C
- Hintergrundbeleuchtung: LED

## 4.10 Kamera-Modul

Eine digitale Kamera wandelt Licht in elektrische Signale um. Zentrales Bauteil ist ein Bildsensor, meist ein CMOS-Sensor (Complementary Metal-Oxide Semiconductor). Der Sensor besteht aus Millionen lichtempfindlicher Pixel. Trifft Licht auf einen Pixel, erzeugt der photoelektrische Effekt eine elektrische Ladung, deren Stärke proportional zur Lichtintensität ist. Diese analogen Signale werden anschließend digitalisiert und als Bilddaten verarbeitet (Szeliski, 2022).

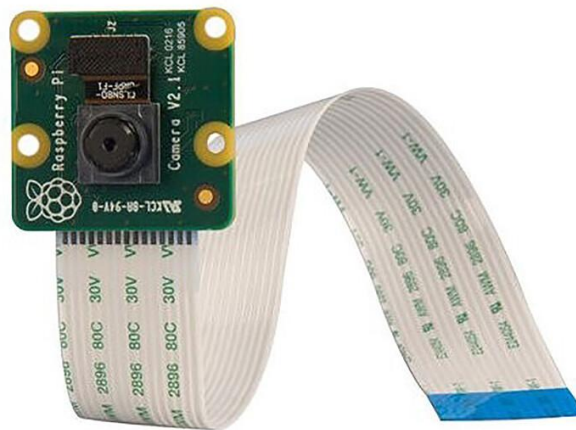


Abbildung 15: Raspberry-Pi-Kameramodul mit CSI-Anschluss (Szeliski, 2022)

### Raspberry Pi Kamera-Modul

Im Smart-Door-Projekt wird ein Raspberry-Pi-Kameramodul verwendet OV5647. Dieses wird über die CSI-Schnittstelle (Camera Serial Interface) direkt mit dem Raspberry Pi verbunden.

Die CSI-Schnittstelle ermöglicht:

- Hohe Datenübertragungsraten
- Direkte Hardware-Anbindung

## Smart Door System

- Geringe CPU-Belastung

### Technische Eigenschaften

- Auflösung: 8 Megapixel
- Video: 1080p
- Schnittstelle: CSI
- Betriebsspannung: 3,3 V

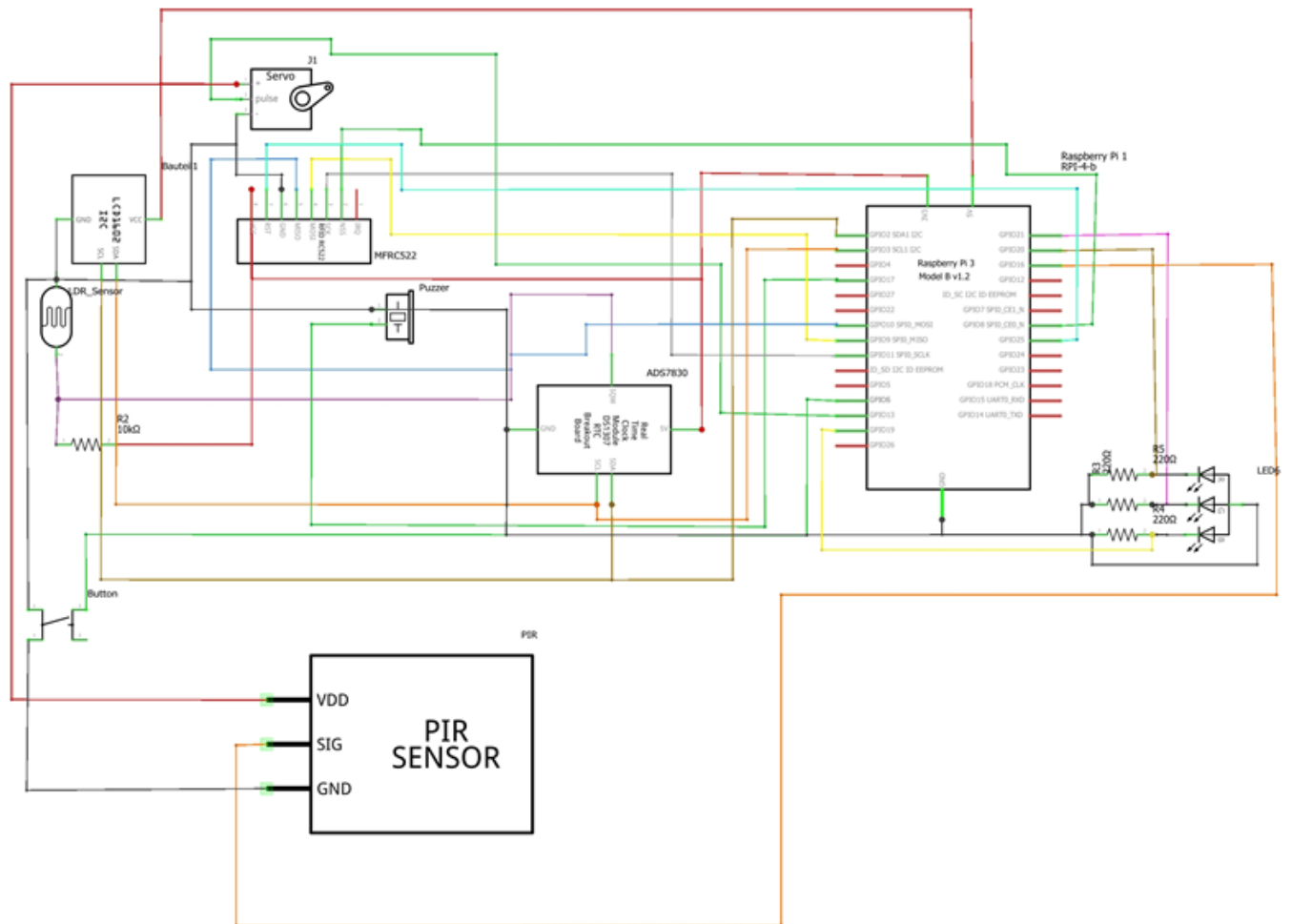
### Funktion im Smart-Door-System

Die Kamera übernimmt im Projekt mehrere Aufgaben:

- Aufnahme eines Fotos bei Klingelereignis
- Dokumentation unautorisierter Zugriffsversuche
- Speicherung der Bilder lokal
- Anzeige im Webinterface

## 5. Schaltungsdesign und Verdrahtung

In diesem Kapitel wird die elektrische Verdrahtung des Smart-Door-Systems beschrieben. Der Raspberry Pi fungiert als zentrale Steuereinheit und verbindet Sensoren (z. B. PIR, LDR/ADC, RFID) sowie Aktoren (Servo, RGB-LED, Buzzer, Display) über GPIO, I<sup>2</sup>C und SPI. Ziel ist eine stabile Signalübertragung, eine sichere Spannungsversorgung und eine wartungsfreundliche Struktur.



fritzing

Abbildung 16: Vollständiger Schaltplan des Smart-Door-Systems

Abbildung 16 zeigt die vollständige elektrische Verschaltung aller Sensoren und Aktoren mit dem Raspberry Pi.

## 5.1 Stromversorgungskonzept

Die Versorgung ist in Smart-Door-Systemen kritisch, weil Servo + RGB-LED + Kamera Lastspitzen erzeugen können. Für zuverlässigen Betrieb muss die Versorgungsspannung des Raspberry Pi stabil bleiben; in der offiziellen Dokumentation wird betont, dass die Versorgung über 4,8 V liegen sollte, um Störungen zu vermeiden.

### Grundstruktur im Projekt:

- Raspberry Pi über stabile 5 V-Versorgung (Netzteil)
- 3,3 V aus dem Raspberry Pi nur für Logik/Sensoren (z. B. PIR je nach Modul, I<sup>2</sup>C-Module)
- Servo idealerweise nicht über 3,3 V betreiben (typisch 5 V), dabei aber:
  - GND gemeinsam verbinden (Pi-GND ↔ Servo-GND)
  - Signal bleibt 3,3 V vom GPIO (PWM)

## 5.2 Sicherheitshinweise bei der Verdrahtung

Für einen zuverlässigen und sicheren Betrieb des Smart-Door-Systems ist eine fachgerechte elektrische Verdrahtung essenziell. Da der Raspberry Pi mit einer Logikspannung von 3,3 V arbeitet, dürfen die GPIO-Pins ausschließlich mit kompatiblen Spannungspegeln betrieben werden. Eine direkte Einspeisung von 5 V in einen GPIO-Pin kann zu dauerhaften Schäden an der Hardware führen. Darüber hinaus sind GPIO-Pins primär für die Signalsteuerung vorgesehen und nicht zur Versorgung leistungsstarker Verbraucher geeignet. Komponenten wie LEDs benötigen geeignete Vorwiderstände, während Motoren oder Servoantriebe nicht direkt über GPIO-Pins betrieben werden dürfen. Diese Bauteile müssen über eine separate Spannungsversorgung angesteuert werden, wobei das Steuersignal vom GPIO lediglich die Regelung übernimmt.

Ein weiterer zentraler Aspekt ist die gemeinsame Masseführung (GND). Alle angeschlossenen Module müssen eine gemeinsame Referenzmasse besitzen, da sonst undefinierte Signalpegel entstehen können. Fehltrigger bei Sensoren (z. B. PIR) oder Kommunikationsfehler bei I<sup>2</sup>C- und SPI-Bussen sind typische Folgen einer unzureichenden Masseverbindung.

Besondere Aufmerksamkeit erfordert die Leitungsführung bei Aktoren wie Servomotoren. Diese können aufgrund ihrer Stromspitzen elektromagnetische Störungen verursachen. Daher sollten Versorgungsleitungen möglichst kurzgehalten und eine saubere Masseführung sichergestellt werden. Gegebenenfalls kann eine zusätzliche Pufferung der 5-V-Versorgung durch Elektrolytkondensatoren in der Nähe des Servos zur Spannungsstabilisierung beitragen.

Um eine strukturierte und sichere Verdrahtung zu gewährleisten, wurden im Projekt Siemens-Reihenklemmen eingesetzt. Diese ermöglichen eine saubere,

## Smart Door System

übersichtliche und mechanisch stabile Verbindung der Leitungen. Zudem erleichtern sie Wartungsarbeiten sowie spätere Erweiterungen des Systems erheblich. Vor jeglichen Umbau- oder Steckarbeiten sollte das System vollständig spannungsfrei geschaltet werden, um Kurzschlüsse und Fehlzustände auf den Kommunikationsleitungen (I<sup>2</sup>C, SPI) zu vermeiden.

## 6. Softwarearchitektur und Implementierung

Die Software bildet das zentrale Steuerungselement des Smart-Door-Systems. Sie übernimmt die Verarbeitung von Sensordaten, die Steuerung der Aktoren sowie die Kommunikation mit externen Schnittstellen wie dem Webserver und Telegramm. Das System wurde in Python implementiert und läuft lokal auf dem Raspberry Pi unter einem Linux-basierten Betriebssystem. Durch die modulare Struktur der Software wird eine klare Trennung zwischen Hardwarezugriff, Logikverarbeitung und Kommunikationsschnittstellen erreicht.

### 6.1 Systemarchitektur

Die Softwarearchitektur folgt einem modularen Aufbau und kann in vier Hauptkomponenten unterteilt werden:

#### Hardware-Abstraktionsschicht

- GPIO-Zugriff (PIR, Buzzer, Servo, RGB-LED)
- I<sup>2</sup>C-Kommunikation (LCD, ADS7830)
- SPI-Kommunikation (RFID)
- Kamera-Ansteuerung

#### Logikschicht

- Ereignisverarbeitung
- Entscheidungslogik (Zugriff erlaubt / verweigert)
- Zeitsteuerung (Cooldown, Tür-Timeout)

#### Kommunikationsschicht

- Lokaler Webserver
- Telegramm-Bot

#### Speicherschicht

- Bildspeicherung
- Log-Management

Die Architektur ist bewusst lokal aufgebaut, um eine Cloud-Abhängigkeit zu vermeiden und die Datensicherheit zu erhöhen.

### 6.2 Ereignisbasierte Logik

Das System arbeitet ereignisorientiert (event-Driven). Das bedeutet, dass keine kontinuierliche Abfrage in festen Zeitintervallen erfolgt, sondern Aktionen nur bei bestimmten Ereignissen ausgelöst werden.

Beispielhafte Ereignisse:

## Smart Door System

- Bewegung erkannt (PIR HIGH)
- RFID-Karte gelesen
- Klingel gedrückt
- Web-Freigabe
- Telegramm-Trigger

Der Ablauf bei Bewegung lässt sich wie folgt beschreiben:

1. PIR erkennt Bewegung
2. Nachtprüfung über LDR
3. Wartezeit für RFID/Klingel
4. Entscheidung:
  - Autorisiert → Tür öffnen
  - Nicht autorisiert → Foto + Meldung

## 6.3 Multithreading

Da mehrere Prozesse parallel laufen müssen, wurde Multithreading eingesetzt.  
Parallellaufende Prozesse:

- Hauptlogik (Sensorüberwachung)
- Webserver
- Telegramm-Kommunikation

Ohne Multithreading würde der Webserver blockieren, während auf RFID oder Sensorereignisse gewartet wird.

## 6.4 Webserver-Implementierung

Zur lokalen Steuerung wurde ein integrierter Webserver implementiert (z. B. Flask oder integrierter HTTP-Server).

Der Webserver bietet:

- Tür öffnen / schließen
- Freigabe bei Klingel
- Statusanzeige
- Anzeige letzter Fotos
- Log-Ausgabe

Die Kommunikation erfolgt über REST-ähnliche API-Endpunkte:

- /api/status
- /api/door/open
- /api/door/close
- /api/allow

## Smart Door System

Die Oberfläche läuft lokal unter <http://<Raspberry-IP>:8080> und Es wurde eine Passwortabfrage implementiert, um unbefugten Zugriff im lokalen Netzwerk zu verhindern. Die Abbildung 17 zeigt den Smart Door Webserver.

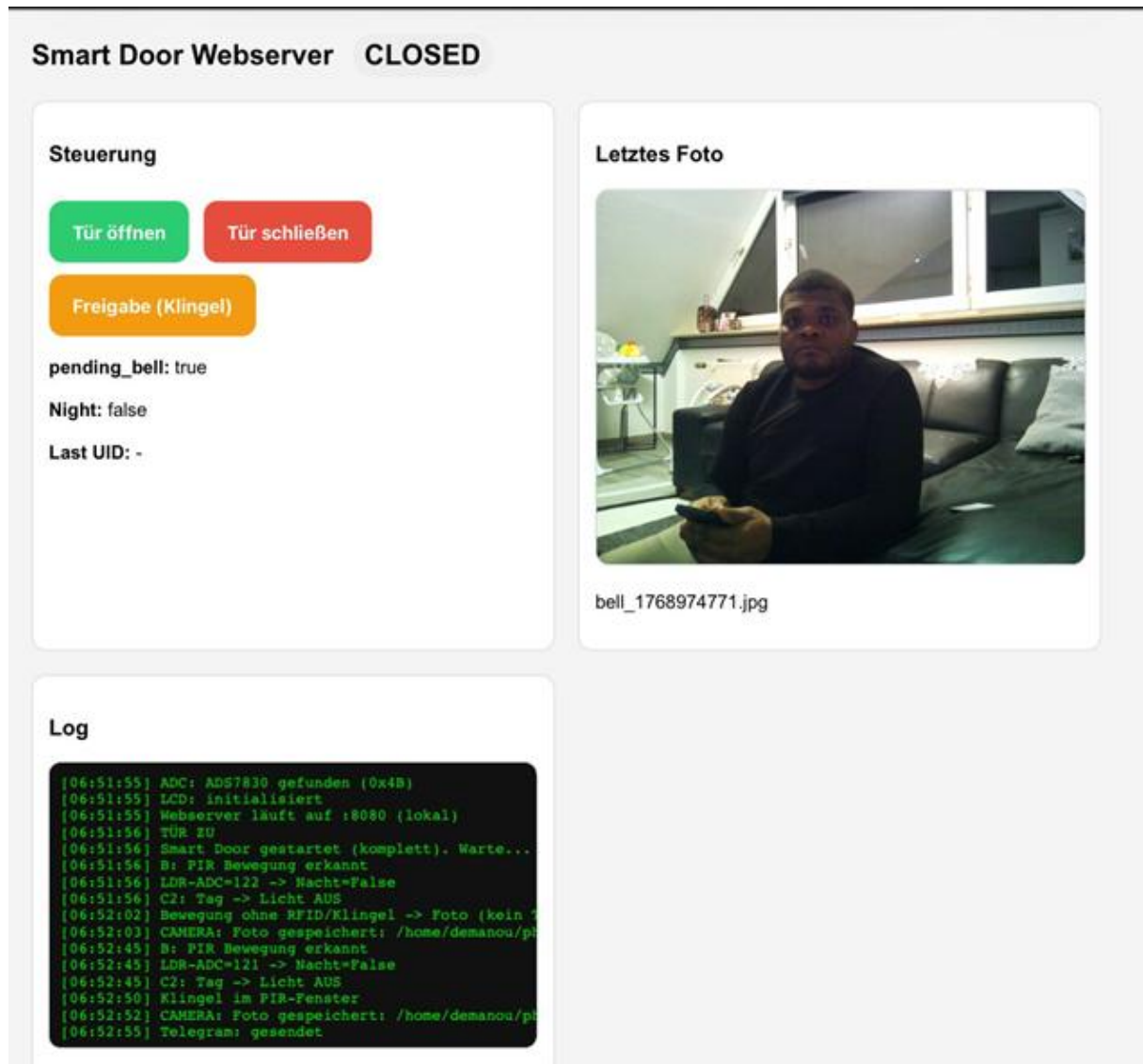


Abbildung 17: Lokales Webinterface des Smart-Door-Systems

Die webbasierte Benutzeroberfläche Abbildung 17 ermöglicht die Überwachung und Steuerung des Systems. Hier werden der aktuelle Türstatus, anstehende Klingelereignisse (`pending_bell`), die Helligkeitserkennung (`Night`) sowie die letzten Systemereignisse im Log angezeigt

## 6.5 Telegram-Integration

Zusätzlich wurde ein Telegramm-Bot integriert, um Benachrichtigungen bei Klingelereignissen zu versenden.

Der Telegramm-Bot Ablauf ist:

1. Klingel gedrückt
2. Foto wird aufgenommen
3. Telegramm-API wird aufgerufen
4. Bild + Nachricht werden an definierte Chat-ID gesendet

Die Kommunikation erfolgt über HTTPS-Anfragen an die Telegramm Bot API.

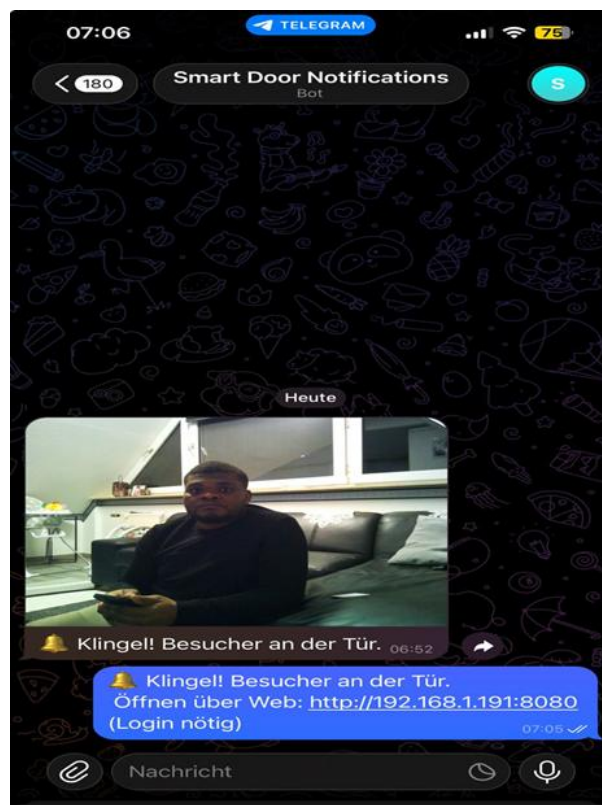


Abbildung 18: Telegram-Benachrichtigung bei Klingelereignis

Die Abbildung 18 zeigt einem Screenshot von Telegram mit Bild + Nachricht

## 7. Funktionsablauf des Systems

In diesem Kapitel wird der vollständige Ablauf des Smart-Door-Systems beschrieben. Die Logik basiert auf einer ereignisorientierten Struktur, bei der Sensoren kontinuierlich überwacht und bei bestimmten Zuständen entsprechende Aktionen ausgelöst werden. Ziel ist eine sichere, automatisierte und nachvollziehbare Zugangskontrolle.

### 7.1 Bewegungserkennung

Der erste Auslöser im System ist die Bewegungserkennung durch den PIR-Sensor. Sobald eine Änderung der Infrarotstrahlung erkannt wird, setzt der Sensor sein Ausgangssignal auf HIGH. Dieses Signal wird vom Raspberry Pi über einen GPIO-Eingang verarbeitet.

Um Fehltrigger durch Störungen oder Prell-Effekte zu vermeiden, erfolgt eine kurze Mehrfachabfrage des Signals. Nur wenn mehrere Messungen hintereinander eine Bewegung bestätigen, wird das Ereignis weiterverarbeitet. Nach erfolgreicher Bewegungserkennung startet die nächste Entscheidungsstufe.

### 7.2 Tag-/Nacht-Entscheidung

Nach der Bewegungserkennung prüft das System über den LDR-Sensor die aktuelle Umgebungshelligkeit. Der analoge Spannungswert wird über den ADS7830-ADC digitalisiert und mit einem definierten Schwellenwert verglichen.

Ist der Messwert oberhalb des Schwellenwertes, wird der Zustand „Nacht“ erkannt. In diesem Fall wird die RGB-LED aktiviert (weißes Licht), um den Eingangsbereich zu beleuchten.

Bei ausreichender Helligkeit bleibt die Beleuchtung deaktiviert.

Diese Entscheidung erhöht sowohl die Energieeffizienz als auch die Benutzerfreundlichkeit.

### 7.3 RFID-Zugangskontrolle

Innerhalb eines definierten Zeitfensters nach Bewegungserkennung wartet das System auf eine RFID-Karte.

Der Ablauf ist wie folgt:

1. RFID-Tag wird an das Lesemodul gehalten
2. UID wird ausgelesen
3. Vergleich mit der autorisierten UID-Liste
4. Entscheidung:

- Autorisierte UID:
  - Türöffnung
  - Grünes LED-Signal
  - Akustische Bestätigung
  - Eintrag im Systemlog
- Nicht autorisierte UID:
  - Tür bleibt geschlossen
  - Rotes LED-Signal
  - Fotoaufnahme
  - Log-Eintrag

Die RFID-Zugangskontrolle bildet die zentrale Sicherheitskomponente des Systems.

### 7.4 Klingel-Logik

Wird der Klingeltaster betätigt, startet das System einen eigenständigen Ablauf, der unabhängig von der Bewegungserkennung durch den PIR-Sensor funktioniert. Dadurch wird sichergestellt, dass Besucher auch dann registriert werden, wenn keine vorherige Bewegung erkannt wurde.

Nach dem Erkennen des Klingelsignals löst das System unmittelbar eine Fotoaufnahme über das Kameramodul aus. Das aufgenommene Bild wird lokal gespeichert und anschließend zusammen mit einer entsprechenden Textnachricht über die Telegramm-Schnittstelle an den definierten Benutzer versendet. Parallel dazu zeigt das lokale Webinterface den Status „Freigabe erforderlich“ an, um eine manuelle Entscheidung zu ermöglichen.

Der Nutzer hat anschließend die Möglichkeit, über das Webinterface die Tür freizugeben. Erfolgt innerhalb eines definierten Zeitfensters keine Freigabe, bleibt die Tür geschlossen und das System kehrt in den Überwachungsmodus zurück. Diese Implementierung erlaubt eine kontrollierte, manuelle Zutrittsentscheidung für nicht registrierte Besucher und erweitert das System um eine flexible Interaktionsmöglichkeit zwischen Benutzer und Zugangskontrolle.

### 7.5 Türsteuerung

Die mechanische Türsteuerung erfolgt über den Servomotor.

Ablauf:

1. Servo wird auf definierte Winkelposition (z. B. 90°) gesetzt
2. Tür bleibt für eine festgelegte Zeit geöffnet
3. Servo fährt zurück auf Ausgangsposition (0°)

Während der Türöffnung werden visuelle (RGB-LED) und akustische (Buzzer) Signale ausgegeben. Nach Ablauf des Zeitintervalls wird der ursprüngliche Zustand wiederhergestellt.

## 8. Mini-Haus als Demonstrationsmodell

Zur realitätsnahen Darstellung des Systems wurde ein Mini-Haus-Modell entwickelt. Dieses dient als physischer Demonstrator der Hardware- und Softwareintegration.

### 8.1 Mechanischer Aufbau

Das Mini-Haus wurde aus 3mm Holz gefertigt und besitzt folgende Elemente:

- Tür mit Servoantrieb
- Ausschnitt für LCD1602
- Öffnung für PIR-Sensor
- Öffnung für LDR
- Platz für Kamera
- LED-Anzeige im Frontbereich
- Klingeltaster

Die Positionierung der Komponenten orientiert sich an realen Türsystemen, um eine praxisnahe Demonstration zu ermöglichen. Die Abbildung 19 zeigt das Bild vom fertigen Mini-Haus Frontansicht.



Abbildung 19: Mini-Haus Demonstrationsmodell (Frontansicht)

## 8.2 Integration der Elektronik

Die gesamte Elektronik befindet sich im Inneren des Modells.

- Raspberry Pi als zentrale Einheit
- Verkabelung über Siemens-Reihenklemmen
- Trennung von Logik- und Versorgungsleitungen
- Saubere Masseführung

Die strukturierte Verdrahtung ermöglicht eine übersichtliche und wartungsfreundliche Integration, wie die Abbildung 20 es zeigt.

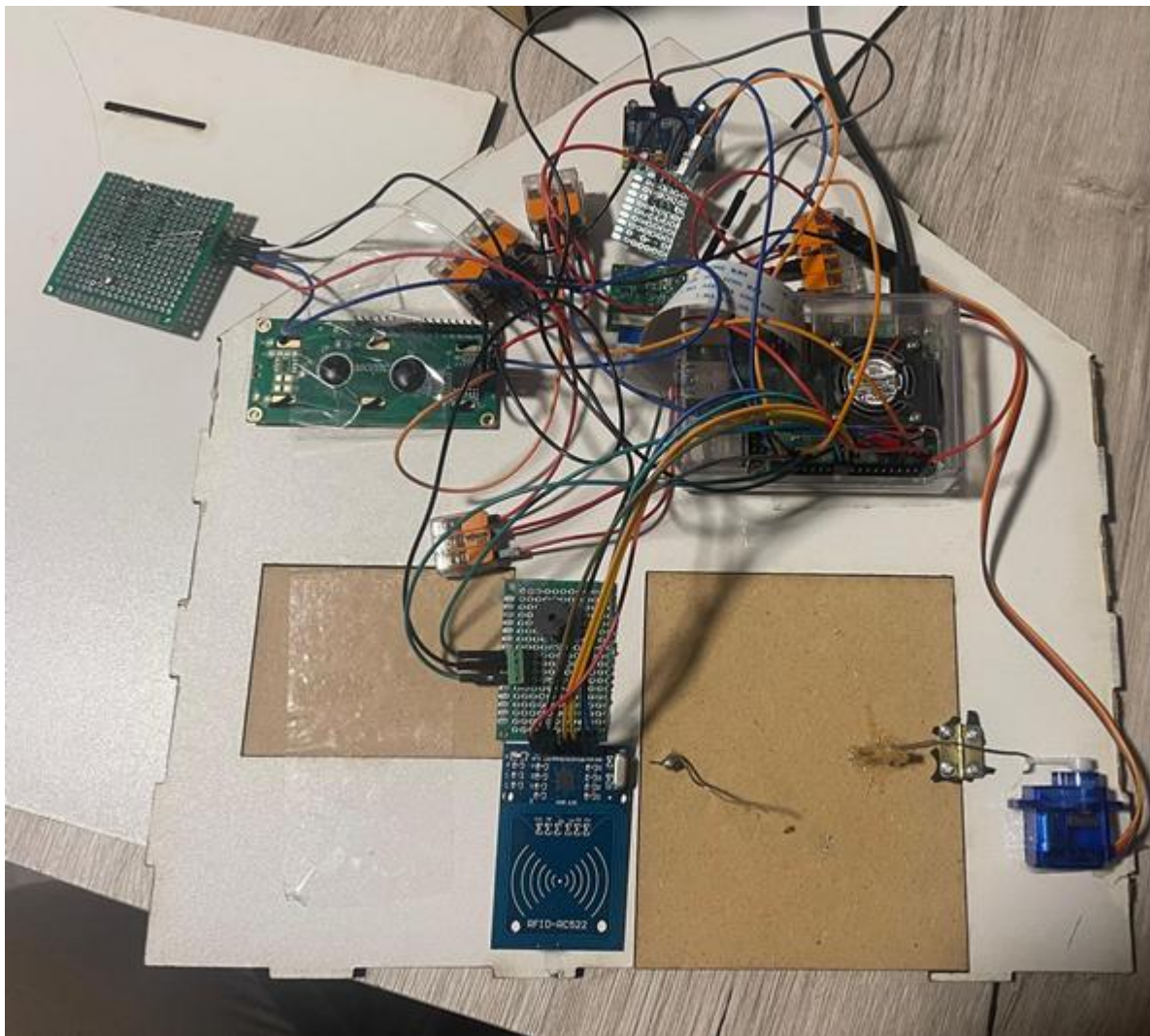


Abbildung 20: Interne Elektronik und Verdrahtung

## 8.3 Herausforderungen beim Aufbau

Während der praktischen Umsetzung des Smart-Door-Systems traten mehrere technische Herausforderungen auf, die sowohl die elektrische als auch die mechanische Integration betrafen.

Eine zentrale Schwierigkeit bestand in den Stromspitzen des Servomotors. Beim Anlaufen oder bei Lastwechseln erzeugte der Servo kurzzeitig erhöhte Stromaufnahmen, die zu leichten Spannungsschwankungen führen konnten. Da das gesamte System – einschließlich Servomotor – direkt über die 5-V-Versorgung des Raspberry Pi betrieben wurde, war eine stabile und saubere Verdrahtung besonders wichtig. Durch eine optimierte Leitungsführung sowie eine saubere Masseverbindung konnte ein stabiler Betrieb ohne zusätzliche externe Spannungsquelle realisiert werden.

Ein weiteres Problem waren Fehltrigger des PIR-Sensors. Insbesondere bei Temperaturschwankungen oder elektrischen Störungen reagierte der Sensor teilweise empfindlich. Dieses Problem wurde durch eine softwareseitige Entprellung sowie durch Mehrfachabfragen des Sensorsignals gelöst, sodass nur bestätigte und stabile Bewegungssignale weiterverarbeitet wurden. Auch die Spannungsstabilität beim parallelen Betrieb mehrerer Komponenten stellte eine Herausforderung dar. Durch eine strukturierte Stromverteilung, kurze Leitungswege und eine klare Trennung zwischen Signal- und Versorgungsleitungen konnte die Zuverlässigkeit deutlich verbessert werden.

Im mechanischen Bereich war die Platzoptimierung innerhalb des Mini-Hauses anspruchsvoll. Da zahlreiche Komponenten auf engem Raum integriert werden mussten, war eine sorgfältige Planung der Anordnung erforderlich. Dabei spielte insbesondere das Kabelmanagement eine wichtige Rolle.

Zur Verbesserung der Übersichtlichkeit und Betriebssicherheit wurden WAGO-Klemmen eingesetzt. Diese ermöglichen eine sichere, werkzeuglose Verbindung der Leitungen und sorgen für eine strukturierte und wartungsfreundliche Verdrahtung. Durch die Verwendung der Klemmen konnten lose Steckverbindungen vermieden und die elektrische Stabilität erhöht werden.

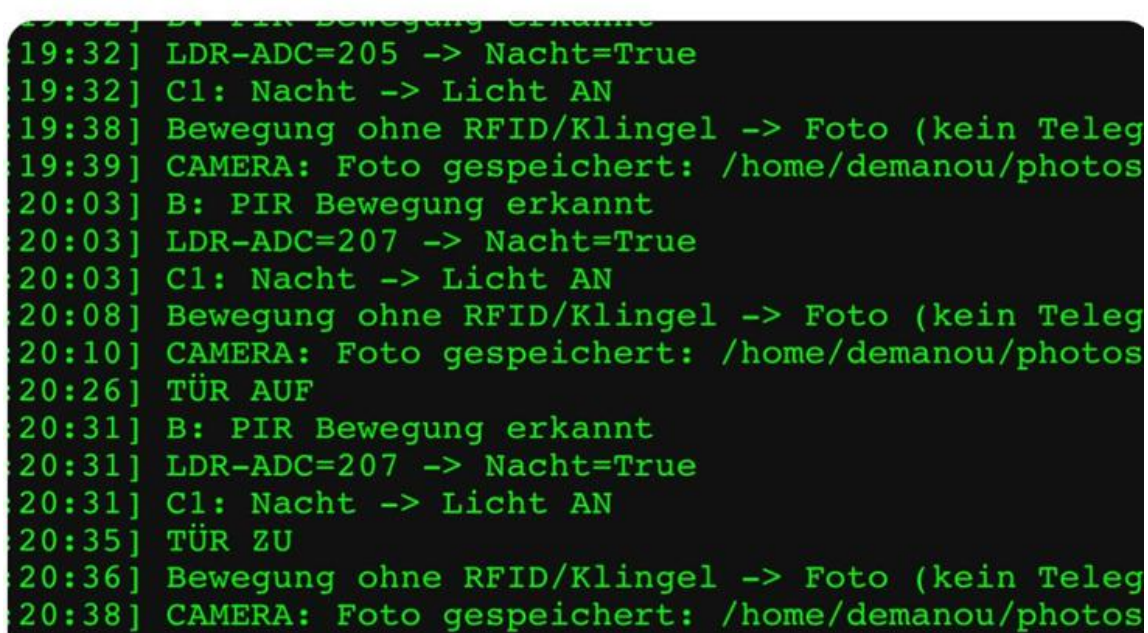
Durch diese Maßnahmen konnten die auftretenden Probleme systematisch analysiert und nachhaltig gelöst werden, wodurch ein stabiler und zuverlässiger Betrieb des Gesamtsystems sichergestellt wurde.

## 9. Testphase und Ergebnisse

Nach Abschluss der Hardware- und Softwareimplementierung wurde eine Testphase durchgeführt. Ziel war es, die Funktionsfähigkeit aller Komponenten zu überprüfen, die Systemstabilität unter realistischen Bedingungen zu analysieren und die Zuverlässigkeit des Gesamtsystems zu bewerten.

Die Tests wurden im aufgebauten Mini-Haus-Modell durchgeführt, wobei sowohl einzelne Komponenten als auch der vollständige Systemablauf überprüft wurden.

### Log



```
19:32] B: PIR Bewegung erkannt  
19:32] LDR-ADC=205 -> Nacht=True  
19:32] C1: Nacht -> Licht AN  
19:38] Bewegung ohne RFID/Klingel -> Foto (kein Teleg  
19:39] CAMERA: Foto gespeichert: /home/demanou/photos  
20:03] B: PIR Bewegung erkannt  
20:03] LDR-ADC=207 -> Nacht=True  
20:03] C1: Nacht -> Licht AN  
20:08] Bewegung ohne RFID/Klingel -> Foto (kein Teleg  
20:10] CAMERA: Foto gespeichert: /home/demanou/photos  
20:26] TÜR AUF  
20:31] B: PIR Bewegung erkannt  
20:31] LDR-ADC=207 -> Nacht=True  
20:31] C1: Nacht -> Licht AN  
20:35] TÜR ZU  
20:36] Bewegung ohne RFID/Klingel -> Foto (kein Teleg  
20:38] CAMERA: Foto gespeichert: /home/demanou/photos
```

Abbildung 21: System-Log mit Bewegungs- und Helligkeitserkennung

In Abbildung 21 ist der System-Log während eines Bewegungsereignisses am Tag dargestellt. Der LDR-Sensor liefert einen ADC-Wert von 122, was zur Erkennung "Nacht=False" führt. Nach der Bewegungserkennung wird ein Foto aufgenommen und bei Betätigung des Klingeltasters eine Telegramm-Benachrichtigung versendet.

### 9.1 Funktionstests der Sensoren

Im ersten Schritt wurden alle Sensoren einzeln getestet.

-PIR-Sensor:

Der PIR-Sensor reagierte zuverlässig auf Bewegungen im Erfassungsbereich von ca. 3–5 Metern. Durch softwareseitige Mehrfachabfrage konnte die Anzahl von

Fehltriggern deutlich reduziert werden. Bei starken Temperaturschwankungen zeigte sich eine leicht erhöhte Empfindlichkeit.

-LDR mit ADS7830:

Der LDR-Sensor lieferte stabile Analogwerte, die über den ADS7830 zuverlässig digitalisiert wurden. Der definierte Schwellenwert für die Tag-/Nacht-Entscheidung funktionierte reproduzierbar. Unterschiede zwischen künstlicher und natürlicher Beleuchtung wurden korrekt erkannt.

-RFID-Modul (MFRC522):

Das RFID-Modul erkannte autorisierte Karten innerhalb einer Distanz von ca. 2–4 cm. Die UID-Auslesung erfolgte ohne Verzögerung. Nicht autorisierte Karten wurden korrekt abgelehnt.

-Klingeltaster:

Der Klingeltaster wurde mehrfach getestet. Die Erkennung erfolgte stabil, und die anschließende Fotoaufnahme sowie Telegramm-Benachrichtigung funktionierten zuverlässig.

-Kamera

Die Kamera nahm bei Ereignissen (Klingel oder unautorisierter Zugriff) erfolgreich Fotos auf. Die Speicherung sowie Anzeige im Webinterface funktionierten ohne Fehler.

## 9.2 Stabilität des Systems

Zur Überprüfung der Systemstabilität wurde das Smart-Door-System über einen längeren Zeitraum im Dauerbetrieb getestet.

Dabei wurden folgende Punkte analysiert:

- Gleichzeitiger Betrieb mehrerer Komponenten
- Parallele Nutzung von Webserver und Sensorüberwachung
- Wiederholte Türöffnungen
- Mehrfaches Senden von Telegramm-Nachrichten

Das System zeigte im Testbetrieb eine stabile Funktion. Auch unter paralleler Belastung durch Multithreading traten keine Systemabstürze oder Deadlocks auf. Die Spannungsversorgung über den Raspberry Pi erwies sich als ausreichend für den Demonstrationsbetrieb. Trotz kurzzeitiger Stromspitzen des Servomotors blieb das System funktionsfähig.

### 9.3 Bewertung der Zuverlässigkeit

Die Zuverlässigkeit des entwickelten Smart-Door-Systems kann insgesamt als hoch bewertet werden. Während der Testphase arbeiteten alle eingesetzten Sensoren zuverlässig und lieferten stabile Messergebnisse. Die Zugriffskontrolle über das RFID-Modul funktionierte korrekt, wobei autorisierte Karten eindeutig erkannt und nicht autorisierte Zugriffe zuverlässig abgewiesen wurden. Falsche Positiv- oder Negativerkennungen traten im Testbetrieb nicht auf.

Auch die Kommunikationsschnittstellen, insbesondere die Webserver-Anbindung sowie die Telegramm-Benachrichtigungen, zeigten eine stabile und fehlerfreie Funktion. Die durchschnittliche Reaktionszeit des Systems lag unter einer Sekunde, wodurch eine nahezu verzögerungsfreie Ereignisverarbeitung gewährleistet wurde. Gelegentliche Fehltrigger des PIR-Sensors konnten durch softwareseitige Mehrfachabfrage und Entprellmechanismen deutlich reduziert werden. Insgesamt erfüllt das System somit die Anforderungen eines funktionsfähigen Demonstrationsprototyps und zeigt eine zuverlässige und stabile Gesamtperformance.

## 10. Sicherheits- und Datenschutzaspekte

Da das System Bildaufnahmen verarbeitet und Zugangsdaten speichert, spielen Sicherheits- und Datenschutzaspekte eine wichtige Rolle.

Folgende Maßnahmen wurden umgesetzt:

- Lokale Verarbeitung ohne Cloud-Anbindung
- Speicherung der Bilder ausschließlich auf dem Raspberry Pi
- Passwortschutz für das Webinterface
- Telegramm-Benachrichtigungen nur bei Klingelereignissen
- Keine permanente Videoüberwachung

Durch die lokale Architektur wird das Risiko externer Datenabflüsse minimiert.

Dennoch sollte bei einem produktiven Einsatz eine verschlüsselte Webkommunikation (HTTPS) sowie eine stärkere Authentifizierung implementiert werden.

## 11.Fazit

Ziel des Projekts war die Entwicklung eines intelligenten, lokal arbeitenden Smart-Door-Systems auf Basis eines Raspberry Pi.

Das entwickelte System kombiniert Bewegungserkennung, Tag-/Nacht-Steuerung, RFID-Zugangskontrolle, Websteuerung und Telegramm-Benachrichtigung in einer modularen Architektur.

Die Testphase bestätigte die Funktionsfähigkeit aller Komponenten. Die Integration in ein Mini-Haus-Modell ermöglichte eine praxisnahe Demonstration des Gesamtsystems.

Insgesamt konnte ein stabiler, funktionsfähiger Prototyp realisiert werden, der sowohl technische als auch sicherheitsrelevante Anforderungen erfüllt.

## 12. Ausblick und mögliche Erweiterungen

Trotz der erfolgreichen Umsetzung bestehen mehrere Erweiterungsmöglichkeiten:

- Integration eines Fingerabdrucksensors
- Einsatz eines präziseren digitalen Lux-Sensors
- Verwendung einer separaten Spannungsversorgung für leistungsstarke Aktoren
- Implementierung von HTTPS-Verschlüsselung
- Mobile App mit direkter Push-Funktion
- Gesichtserkennung mittels KI
- Integration in bestehende Smart-Home-Plattformen

Eine Weiterentwicklung in Richtung eines seriennahen Systems würde insbesondere eine robustere mechanische Konstruktion sowie eine normgerechte elektrische Absicherung erfordern.

## Literaturverzeichnis

- AMS (TSL2561 Datasheet). (2023). Von [https://ams.com/documents/20143/36005/TSL2561\\_DS000110\\_2-00.pdf](https://ams.com/documents/20143/36005/TSL2561_DS000110_2-00.pdf) abgerufen
- Atzori, L., Lera, A., & Morabito, G. (2010). The Internet of Things: A Survey. *IEEE Communications Magazine*, 48(7), S. 102-108.  
doi:10.1109/MCOM.2010.5514475
- Bolton W. (2015). *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*. Pearson.
- ENISA- Datenschutz & IoT. (2021). Von [www.enisa.europa.eu](http://www.enisa.europa.eu). abgerufen
- ESP32, T. R. (2023). [www.espressif.com](http://www.espressif.com). Von <https://www.espressif.com/en/products/socs/esp32> abgerufen
- Finkenzeller, K. (2010). *RFID Handbook*. Wiley.
- Hill, H. &. (2015). *The Art of Electronics (3rd ed.)*. . Cambridge University.
- Hitachi (HD44780U Datasheet). (kein Datum). [www.sparkfun.com](http://www.sparkfun.com). Von <https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf> abgerufen
- Kingbright RGB LED Datasheet. (kein Datum). [www.kingbrightusa.com](http://www.kingbrightusa.com). Von <https://www.kingbrightusa.com/images/catalog/spec/WP154A4SEJ3VBDZGW.pdf> abgerufen
- Lee, E. A. (11 2008). [ieeexplore.ieee.org](http://ieeexplore.ieee.org). Von <https://doi.org/10.1109/ISORC.2008.25> abgerufen
- Michael Barr, A. M. (2006). *Programming Embedded Systems, 2nd Edition*. O'Reilly Media, Inc.
- Motorola. (2003). [www.nxp.com](http://www.nxp.com). Von <https://www.nxp.com/docs/en/reference-manual/S12SPIV3.pdf> abgerufen

- National Institute of Standards and Technology (NIST). (2019). Von <https://nvlpubs.nist.gov/nistpubs/ir/2019/NIST.IR.8228.pdf> abgerufen
- NXP Semiconductors (MFRC522 Datasheet). (2023). Von <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf> abgerufen
- NXP Semiconductors. (2021). *www.nxp.com*. Von <https://www.nxp.com/docs/en/user-guide/UM10204.pdf> abgerufen
- Ptruzzellis T. (2017). *Electronic Devices and Circuits*. McGraw-Hill.
- Raspberry pi Foundation. (2023). *raspberrypi.com*. Von <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#gpio> abgerufen
- ROHM (BH1750 Datasheet). (2026). *www.rohm.com*. Von <https://www.rohm.com/datasheet/BH1750FVI/bh1750fvi-e> abgerufen
- Stromberg, G. (2019). *Smart Home-Grundlagen, Technik und Anwendungen*. Springer Vieweg.
- Szeliski, R. (2022). *Computer Vision: Algorithms and Applications, 2nd ed.* Springer.
- Texas Instruments (ADS7830 Datasheet). (2023). *www.ti.com*. Von <https://www.ti.com/lit/ds/symlink/ads7830.pdf> abgerufen
- TowerPro SG90 Datasheet. (2021). *www.components101.com*. Von [https://components101.com/sites/default/files/component\\_datasheet/SG90%20Servo%20Motor%20Datasheet.pdf](https://components101.com/sites/default/files/component_datasheet/SG90%20Servo%20Motor%20Datasheet.pdf) abgerufen
- Wyszeck G. & Stiles W. (2000). *Color Science: Concepts and Methods*. Wiley.

## 13. Anhang

```
# config.py
from dataclasses import dataclass

@dataclass(frozen=True)
class GPIOPins:
    PIR: int = 16
    SERVO: int = 13
    BUZZER: int = 17
    BUTTON_BELL: int = 6

    LED_R: int = 19
    LED_G: int = 20
    LED_B: int = 21

@dataclass(frozen=True)
class I2CConfig:
    BUS: int = 1
    ADC_ADDR: int = 0x4B      # ADS7830
    LCD_ADDR: int = 0x27     # häufig 0x27 oder 0x3F (bei dir prüfen)

@dataclass(frozen=True)
class WebConfig:
    HOST: str = "0.0.0.0"
    PORT: int = 8080
    USERNAME: str = "admin"
    PASSWORD: str = "admin"  # im Produktivbetrieb ändern

@dataclass(frozen=True)
class TelegramConfig:
    BOT_TOKEN: str = "PASTE_YOUR_TOKEN_HERE"
    CHAT_ID: str = "PASTE_YOUR_CHAT_ID_HERE"

@dataclass(frozen=True)
class SystemConfig:
    NIGHT_THRESHOLD: int = 210    # ADC (0..255) -> Schwellenwert anpassen
    RFID_TIMEOUT_S: int = 8
    DOOR_OPEN_S: int = 3
    PIR_COOLDOWN_S: int = 5
```

### Anhang1: Konfiguration & GPIO-Zuordnung

## Smart Door System

```
# adc_ads7830.py
from smbus2 import SMBus

class ADS7830:
    """
    ADS7830: 8-bit ADC via I2C.
    Liefert Werte 0..255.
    """
    def __init__(self, bus_id: int, addr: int = 0x4B):
        self.addr = addr
        self.bus = SMBus(bus_id)

    def read_channel(self, ch: int = 0) -> int:
        """
        ch: 0..7
        Command: Single-ended, internal ref, power-down between conversions.
        """
        if not (0 <= ch <= 7):
            raise ValueError("Channel must be 0..7")

        # ADS7830 command byte (TI Datasheet): 1 0 0 0 | C2 C1 C0 | PD1 PD0
        # Single-ended channel selection uses a specific mapping; häufig genutzt:
        # cmd = 0x84 | (((ch << 2) | (ch >> 1)) & 0x07) << 4 (variiert je nach wiring)
        # Praxis: viele Beispiele nutzen:
        cmd = 0x84 | ((ch & 0x07) << 4)

        self.bus.write_byte(self.addr, cmd)
        return self.bus.read_byte(self.addr)

    def close(self):
        self.bus.close()
```

Anhang2: ADS7830 lesen (LDR → Tag/Nacht)

## Smart Door System

```
# telegram_notify.py
import requests

class TelegramNotifier:
    def __init__(self, token: str, chat_id: str):
        self.base = f"https://api.telegram.org/bot{token}"
        self.chat_id = chat_id

    def send_text(self, text: str):
        r = requests.post(f"{self.base}/sendMessage", data={"chat_id": self.chat_id, "text": text},
            r.raise_for_status()

    def send_photo(self, photo_path: str, caption: str = ""):
        with open(photo_path, "rb") as f:
            files = {"photo": f}
            data = {"chat_id": self.chat_id, "caption": caption}
            r = requests.post(f"{self.base}/sendPhoto", data=data, files=files, timeout=20)
            r.raise_for_status()
```

### Anhang3: Telegram Nachricht + Foto senden

## Smart Door System

```
# main.py
import time
import threading
from pathlib import Path

import RPi.GPIO as GPIO

from config import GPIOPins, I2CConfig, WebConfig, TelegramConfig, SystemConfig
from adc_ads7830 import ADS7830
from telegram_notify import TelegramNotifier
from webserver import create_app

# OPTIONAL: Kamera - je nach Setup picamera2 / libcamera
# Hier nur als Platzhalter-Funktion implementiert:
def take_photo(save_dir: Path, prefix: str) -> Path:
    save_dir.mkdir(parents=True, exist_ok=True)
    ts = int(time.time())
    path = save_dir / f"{prefix}_{ts}.jpg"
    # TODO: hier deinen Kamera-Aufruf eintragen (libcamera-still oder picamera2)
    # Beispiel (Shell): os.system(f"libcamera-still -o {path} -n --width 640 --height 480")
    return path

class Actions:
    def __init__(self, pins: GPIOPins, cfg: SystemConfig, state: dict):
        self.pins = pins
        self.cfg = cfg
        self.state = state
        self._bell_allowed_until = 0.0

        # Servo PWM (50 Hz)
        self.servo_pwm = GPIO.PWM(self.pins.SERVO, 50)
        self.servo_pwm.start(0)

    def _servo_angle(self, angle: float):
        # SG90 typisch: 0°~2.5% duty, 90°~7.5%, 180°~12.5% (grobe Faustregel)
        duty = 2.5 + (angle / 180.0) * 10.0
        self.servo_pwm.ChangeDutyCycle(duty)
        time.sleep(0.4)
        self.servo_pwm.ChangeDutyCycle(0)
```

## Anhang4: Hauptlogik

```
from flask import Flask, jsonify, request, Response
from functools import wraps
from config import WebConfig

def basic_auth_required(username: str, password: str):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            auth = request.authorization
            if not auth or auth.username != username or auth.password != password:
                return Response("Login nötig", 401, {"WWW-Authenticate": 'Basic realm="Door"'})
            return fn(*args, **kwargs)
        return wrapper
    return decorator

def create_app(state, actions, web_cfg: WebConfig):
    """
    state: dict-like (shared) -> Statuswerte
    actions: object mit Methoden open_door(), close_door(), allow_bell()
    """
    app = Flask(__name__)
    auth = basic_auth_required(web_cfg.USERNAME, web_cfg.PASSWORD)

    @app.get("/api/status")
    @auth
    def status():
        return jsonify(state)

    @app.post("/api/door/open")
    @auth
    def door_open():
        actions.open_door()
        return jsonify({"ok": True})

    @app.post("/api/door/close")
    @auth
    def door_close():
        actions.close_door()
        return jsonify({"ok": True})
```